



FACULTAD DE TECNOLOGÍAS INTERACTIVAS

Servicio Backend para la Gestión de la Variabilidad en Líneas de Productos Educativos

Informe Técnico de la asignatura de Proyecto de Investigación y Desarrollo V

Autor(es): Jose Luis Echemendia López

Tutor(es): M. Sc. Yadira Ramírez Rodríguez, Ing. Liany Sobrino Miranda

La Habana, Diciembre de 2025

“Año 67 de la Revolución”

Resumen

En el contexto de la rápida evolución tecnológica, la Educación Superior enfrenta el desafío de diseñar Planes de Estudio flexibles que respondan a la demanda de perfiles profesionales especializados. Esta investigación aborda este problema aplicando los principios de la Ingeniería de Líneas de Productos de Software (SPL) y los Modelos de Características (Feature Models - FM) al dominio de la planificación curricular.

El trabajo se centra en el diseño, implementación y validación de un *backend* que permita a los gestores académicos modelar currículos como "Líneas de Productos Educativos". El sistema implementa una API RESTful que sirve como núcleo para las operaciones de creación, análisis, validación y configuración de FMs. La arquitectura del servidor gestiona la lógica de negocio para representar características y relaciones de obligatoriedad, optatividad y prerrequisitos como restricciones formales, garantizando la consistencia e integridad de los datos mediante un motor de reglas eficiente y un modelo de datos optimizado.

El resultado principal es un servicio backend especializado que no solo soporta el modelado y la persistencia de FMs adaptados al ámbito educativo, sino que también proporciona la base computacional para automatizar la generación y validación de Planes de Estudio diversos y consistentes a partir de un tronco común. Este desarrollo sienta las bases técnicas para una nueva generación de herramientas de gestión académica, demostrando la viabilidad de transferir técnicas avanzadas de ingeniería de software al dominio educativo.

Palabras claves: Modelos Característicos, Líneas de Productos de Software, Líneas de Productos Educativos, Variabilidad, Motor de Reglas, Plan de Estudios.

Abstract

In the context of rapid technological evolution, Higher Education faces the challenge of designing flexible Study Plans that respond to the demand for specialized professional profiles. This research addresses this problem by applying the principles of Software Product Line Engineering (SPL) and Feature Models (FM) to the domain of curriculum planning.

The work focuses on the design, implementation, and validation of a backend that allows academic managers to model curricula as "Educational Product Lines". The system implements a RESTful API that serves as the core for the creation, analysis, validation, and configuration of FMs. The server architecture manages the business logic to represent subjects as features and relationships of obligation, optionality, and prerequisites as formal constraints, ensuring data consistency and integrity through an efficient rule engine and an optimized data model.

The main result is a specialized backend service that not only supports the modeling and persistence of FMs adapted to the educational field but also provides the computational basis for automating the generation and validation of diverse and consistent Study Plans from a common trunk. This development lays the technical foundations for a new generation of academic management tools, demonstrating the feasibility of transferring advanced software engineering techniques to the educational domain.

Keywords: *Feature Models, Software Product Lines, Educational Product Lines, Variability, Rule Engine, Curriculum.*

Tabla de Contenidos

Introducción	1
1 Desarrollo	8
Introducción	8
1.1 Fundamentación Teórica	8
1.1.1 Conceptos asociados al tema	9
1.2 Análisis del mercado	24
1.2.1 Panorama General del Mercado	24
1.2.2 Análisis de Soluciones Existentes	24
1.2.3 Oportunidades de Mercado Identificadas	27
1.3 Metodología de desarrollo del software	29
1.3.1 Metodologías de desarrollo de software tradicionales	30
1.3.2 Metodologías de desarrollo de software ágiles	31
1.3.3 Elección de la metodología	32
1.3.4 Programación Extrema	35
1.4 Herramientas y tecnologías	39
1.4.1 Lenguaje de Programación	39
1.4.2 Framework de Desarrollo	42
1.4.3 Base de Datos y Gestor de Base de Datos	44
1.4.4 Contenerización y Orquestación	46
1.4.5 Almacenamiento en Memoria, Caching y Mensajería	48
1.4.6 Almacenamiento de Archivos y Objetos	50
1.4.7 Procesamiento Asíncrono	51
1.4.8 Algoritmos, Motor de Reglas y Validación	55
1.4.9 Gestor de dependencias	58

1.4.10 Herramienta para el control de versiones	59
1.4.11 Lenguaje de modelados	60
1.4.12 Herramienta CASE	61
1.4.13 Herramientas para diseñar y ejecutar pruebas	62
1.4.14 Entorno Integrado de Desarrollo	63
1.5 Propuesta de Solución	64
1.5.1 Modelo SPL y su Correspondencia con la Propuesta	65
1.5.2 Arquitectura conceptual	74
1.5.3 Componentes funcionales del sistema	76
1.5.4 Patrones de Diseño	77
1.5.5 Modelo de Datos	82
Conclusiones	98
Acrónimos	100
Glosario	104

Índice de Tablas

1.1 Notación semántica de las relaciones entre características de los Modelo de Características (<i>Feature Model</i>) (FM). Fuente: Elaboración propia.	13
1.2 Comparación, Metodología Ágil VS Metodología Tradicional [Awad, 2005]	33
1.3 Comparación de la Metodología Ágil y la Metodología Tradicional con respecto al proyecto. Fuente: Elaboración propia	34
1.4 Comparación de Metodología Ágiles. Fuente: Elaboración propia	35
1.5 Tabla Comparativa: FastAPI vs FastAPI + Celery. Fuente: Elaboración propia.	54
1.6 Analogía de conceptos de Línea de Productos de Software (<i>Software Product Line</i>) (SPL) en el Dominio Educativo. Fuente: Elaboración propia.	68
1.7 Diccionario de datos: Tabla <code>feature_model</code>	84
1.8 Diccionario de datos: Tabla <code>feature_model_versions</code>	86
1.9 Diccionario de datos: Tabla <code>features</code>	88
1.10Diccionario de datos: Tabla <code>feature_groups</code>	90
1.11Diccionario de datos: Tabla <code>feature_relations</code>	92
1.12Diccionario de datos: Tabla <code>constraints</code>	94
1.13Diccionario de datos: Tabla <code>configurations</code>	95

Índice de Figuras

1.1 Símbolos gráficos para representar las relaciones estructurales. Fuente: Elaboración propia.	14
1.2 Representación simplificada #1 del FM de la asignatura Estructura de Datos I. Fuente: elaboración propia.	70
1.3 Representación simplificada #2 del FM de la asignatura Estructura de Datos I. Fuente: elaboración propia.	71
1.4 Modelo entidad-relación.	83

Introducción

El panorama de la Educación Superior en el siglo XXI está inmerso en una dinámica de cambio sin precedentes por rápidos avances tecnológicos y cambios sociales. La aceleración tecnológica y las fluctuantes demandas del mercado laboral exigen que las instituciones académicas migren de currículos rígidos hacia planes de estudio dinámicos y flexibles [UNESCO, 2022]. Esta necesidad de flexibilidad introduce un desafío crítico: la gestión de la variabilidad curricular. La innovación educativa y la gestión curricular se han convertido en elementos clave para transformar el paradigma educativo y preparar a los estudiantes para un futuro cada vez más digitalizado y globalizado.

La innovación educativa y la gestión curricular son aspectos fundamentales para el desarrollo de sistemas educativos actuales, sin embargo, en el presente, resalta la necesidad de una gestión curricular que promueva la colaboración entre docentes, la adaptación a las necesidades del alumnado y la actualización de los contenidos, métodos pedagógicos, planes de estudios, planificación de asignaturas e incluso, carreras en general.

La innovación educativa se refiere a la introducción de nuevas ideas, enfoques, metodologías y tecnologías en el ámbito educativo; busca mejorar la calidad de la enseñanza y el aprendizaje, fomentando la creatividad, el pensamiento crítico y la resolución de problemas en los estudiantes. La innovación educativa también promueve la adaptación de los procesos educativos a las necesidades cambiantes de la sociedad y el entorno laboral. Por otro lado, la gestión curricular implica el diseño, desarrollo e implementación de planes de estudio y programas educativos. Por otra parte, la gestión curricular se ocupa de organizar y estructurar los contenidos, las habilidades y las competencias que se enseñan en

las instituciones educativas; además, implica la evaluación y la mejora continua de los programas curriculares para asegurar su relevancia y efectividad [Gómez, 2023].

La innovación educativa y la gestión curricular están estrechamente interrelacionadas. La introducción de enfoques innovadores en la enseñanza y el aprendizaje requiere una gestión curricular flexible y adaptable, esto implica revisar y actualizar constantemente los planes de estudio para integrar nuevas metodologías, tecnologías y recursos educativos. Igualmente, la gestión curricular efectiva es fundamental para garantizar que los cambios e innovaciones en el ámbito educativo se implementen de manera coherente y sistemática. La integración de las Tecnologías de la Información y Comunicación (TIC) en el aula permite ampliar el acceso a la información, fomentar la colaboración entre estudiantes, personalizar el aprendizaje y desarrollar habilidades digitales clave, de la misma manera, la gestión curricular puede incluir la planificación y el uso estratégico de la tecnología en los programas educativos para maximizar su impacto y beneficios [Gómez, 2023].

En este sentido, la innovación educativa y la gestión curricular deben fundamentarse en una comprensión profunda de las particularidades y demandas de los estudiantes. La educación contemporánea coloca en el centro la atención a la diversidad, lo que convierte la inclusión y la personalización del aprendizaje en elementos indispensables dentro de cualquier propuesta formativa. Esto supone diseñar y organizar planes de estudio flexibles, capaces de responder a distintas habilidades, ritmos, estilos cognitivos y necesidades específicas que presentan los estudiantes a lo largo de su proceso educativo.

Asimismo, es esencial reconocer que tanto la innovación pedagógica como la gestión curricular no se desarrollan como acciones aisladas o independientes, sino que forman parte de un proceso dinámico y articulado. Su efectividad depende, en gran medida, del trabajo colaborativo y del compromiso conjunto de todos los actores que intervienen en

el ámbito educativo. Directivos, docentes, estudiantes, familias y miembros de la comunidad deben involucrarse activamente para construir entornos de aprendizaje que favorezcan la equidad, la participación y la mejora continua.

De esta manera, la innovación y la gestión curricular no solo buscan transformar prácticas tradicionales, sino también crear condiciones que permitan responder de manera pertinente a los desafíos pedagógicos actuales. Su verdadero impacto se evidencia cuando logran integrar la voz de los estudiantes, promover la corresponsabilidad en la toma de decisiones y generar propuestas educativas que contribuyan al desarrollo integral de cada persona.

La búsqueda de esta flexibilidad ha llevado a la exploración de paradigmas de ingeniería de software aplicados al ámbito educativo. El concepto de SPL ofrece un enfoque estructurado y probado para gestionar familias de productos a partir de un conjunto común de activos [Apel et al., 2013]. Cuando este paradigma se aplica al dominio educativo, surge el concepto de "Líneas de Productos Educativos", donde un tronco curricular común puede derivar en múltiples planes de estudio especializados mediante la gestión explícita de su variabilidad. La investigación en este campo es incipiente pero prometedora; por ejemplo, estudios en el ecosistema de software educativo *SOLAR* han demostrado que la modelación de la variabilidad, incluso en componentes dinámicos como los foros de discusión, es fundamental para crear sistemas adaptativos que respondan a contextos educativos diversos [Guzmán-Luján et al., 2022].

Actualmente, la definición y el mantenimiento de la variabilidad curricular se gestionan principalmente mediante documentos estáticos, hojas de cálculo y procesos manuales, un enfoque que resulta costoso, lento y altamente propenso a errores. Esta dependencia de herramientas fragmentadas y no integradas dificulta la actualización oportuna de la información, genera inconsistencias entre versiones y complica la

trazabilidad de los cambios realizados. Como consecuencia, pueden diseñarse rutas formativas incoherentes o inválidas, afectando la progresión del estudiante y la calidad del proceso educativo. Además, este tipo de gestión incrementa la carga de trabajo administrativo y docente, limita la visibilidad global del currículo, provoca la divulgación de información desactualizada y compromete la coherencia institucional. Todo ello tiene un impacto directo en la satisfacción del estudiante, en los procesos de acreditación y en la capacidad de la institución para adaptarse rápidamente a nuevas demandas educativas o del mercado laboral. En conjunto, estos efectos negativos evidencian que los métodos tradicionales basados en documentación manual ya no son sostenibles, pues ponen en riesgo la calidad, pertinencia y eficiencia del modelo formativo.

La proliferación de modalidades educativas (presenciales, híbridas, en línea, Curso en Línea Masivo y Abierto (*Massive Open Online Course*) (MOOC), *micro-learning*) y enfoques pedagógicos (aprendizaje basado en proyectos, *flipped classroom*, gamificación) ha creado un panorama de extrema variabilidad instruccional [Bates, 2019]. Actualmente, esta complejidad se gestiona predominantemente mediante soluciones *ad-hoc* que resultan en implementaciones rígidas y de alto costo de mantenimiento [Sánchez et al., 2020]. La educación se enfrenta a la difícil actividad de crear experiencias de aprendizaje verdaderamente personalizadas para cada estudiante o contexto institucional sin un esfuerzo manual enorme. Además, cada nuevo curso o recurso educativo se desarrolla casi desde cero, sin reutilizar componentes comunes de manera sistemática.

De este análisis, se desprende la siguiente interrogante que define el **problema de investigación**: ¿Cómo implementar un servicio *backend* que proporcione la funcionalidad completa para la gestión, validación y configuración de FM en el contexto de Líneas de Productos Educativos? Tomando como **objeto de estudio**: Ingeniería de Líneas

de Productos de Software (*Software Product Line Engineering*) (SPLE) aplicada al dominio educativo. Enmarcado en el **campo de acción:** servicios *backend* especializados para la gestión de la variabilidad en Líneas de Productos Educativos. Con el fin de solucionar la problemática planteada, se define como **objetivo general:** desarrollar un servicio *backend* para la gestión de la variabilidad en Líneas de Productos Educativos.

Para dar cumplimiento al objetivo planteado se proponen las siguientes **tareas investigativas:**

1. **Estudio** exhaustivo del estado del arte en Ingeniería de Líneas de Productos de Software, FM y su aplicación al dominio educativo.
2. **Análisis** de diferentes soluciones homólogas para la identificación de características generales y tendencias en este tipo de aplicaciones.
3. **Diseño** de la arquitectura del servicio *backend*, definiendo los componentes principales y el modelo de datos para representar FM en el contexto educativo.
4. **Selección** de las herramientas, tecnologías y metodología a emplear, para garantizar una excelente y eficiente experiencia de desarrollo y alineado con los objetivos del proyecto.
5. **Identificación** de los requisitos funcionales y no funcionales necesarios para un sistema capaz de gestionar la variabilidad de Líneas de Productos Educativos.
6. **Implementación** del servicio *backend* para materializar el sistema que dará respuesta a la problemática planteada, asegurando la creación, análisis, validación y configuración de FM.
7. **Validación** del servicio desarrollado mediante casos de uso representativos en el ámbito educativo, evaluando su funcionalidad,

rendimiento, utilidad y efectividad para capturar la variabilidad en la educación.

8. **Documentación** de los resultados obtenidos y elaboración de recomendaciones para futuras investigaciones en el área.

Para darle solución a los objetivos trazados en las tareas de investigación se emplearon los siguientes **métodos científicos**:

- **Métodos teóricos:**

- **Modelación:** se utilizó para la elaboración de los diagramas del lenguaje de modelado y en la representación de las características de la solución.
- **Histórico-Lógico:** Se empleó para comprender la evolución de las técnicas de gestión de variabilidad en SPL, desde sus orígenes en ingeniería de software hasta su aplicación potencial en dominios educativos, analizando su desarrollo histórico y lógica interna.
- **Analítico-sintético:** Utilizado en el examen crítico de la bibliografía especializada sobre SPL, FM y sistemas educativos, permitiendo sintetizar un marco conceptual adaptado al dominio educativo.

- **Métodos empíricos:**

- **Observación:** Mediante este método se analizaron herramientas existentes de gestión SPL (como *pure::variants*, *FeatureIDE*) para identificar limitaciones técnicas y requisitos necesarios para el contexto educativo específico.
- **Entrevista:** Se entrevista especialistas responsables de diseños curriculares que compran aspectos técnicos para identificar los desafíos existentes en la flexibilidad y variabilidad de los currículos.

- **Experimentación:** Mediante la implementación de prototipos funcionales para validar diferentes enfoques arquitectónicos y algoritmos de validación.

1. Desarrollo

Introducción

Este espacio tiene como objetivo principal argumentar las bases de la investigación, lo que conlleva a un entendimiento mayor sobre el problema a resolver. Se estudiará el mercado actual para identificar sistemas que empleen el concepto de variabilidad de líneas de productos aplicados a modelos característicos y así determinar la viabilidad del proyecto, posibles características a asumir de la competencia y a su vez diferenciarnos de la misma. Se darán a conocer las herramientas con las cuales se desarrollará el sistema y el por qué ellas serán las protagonistas; lo mismo se dirá del entorno y la metodología de desarrollo. Por último, se tendrá una descripción de cómo se debe comportar el sistema y de qué se espera de él, cuya evidencia de que se está cumpliendo el objetivo serán las primeras pinceladas del sistema.

1.1. Fundamentación Teórica

La base teórica funciona como el almacén conceptual sobre el cual se erige todo trabajo investigativo. Esta sección delimita las nociones esenciales, los fundamentos teóricos existentes y los antecedentes modelares que respaldan la indagación, brindando un sistema de referencia que guía la captación y interpretación de los datos. Este epígrafe establece el marco conceptual necesario para comprender la propuesta de solución. Se parte de los conceptos generales de la Ingeniería de Software para luego enfocarse en una técnica específica, modelos característicos, para finalmente concluir la transición del conocimiento

hacia el dominio de la planificación y variabilidad educativa.

1.1.1 Conceptos asociados al tema

En este epígrafe se desglosan los conceptos esenciales que convergen en el núcleo del estudio: desde los fundamentos literarios y el valor pedagógico de la fábula, pasando por los desafíos y oportunidades de su digitalización, hasta los sistemas tecnológicos diseñados para su gestión y análisis. La clarificación de estos términos no solo delimita el campo de acción, sino que también proporciona el lenguaje común y la base teórica necesaria para el diseño de una solución coherente y efectiva.

1.1.1.1 Ingeniería de Líneas de Productos de Software (SPL)

La Ingeniería de SPL es una estrategia de ingeniería de software que busca producir familias de sistemas de software similares a partir de un conjunto común de activos centrales mediante un proceso gestionado de desarrollo y gestión de la variabilidad. El objetivo principal es lograr la reutilización a gran escala, lo que se traduce en una reducción de costos, menor tiempo de comercialización y mayor calidad de los productos [SG Buzz]. El concepto de SPL surgió a finales de los años 90 como respuesta directa a la crisis de productividad que enfrentaba la industria del software a pesar de las técnicas de reutilización orientadas a objetos. La SPLE formaliza una práctica que ya existía intuitivamente en muchas empresas: construir un software base (plataforma) para generar rápidamente múltiples productos similares con variaciones controladas [AcademiaLab]

Conceptos Clave de SPL:

- 1. Línea de Productos:** Es la familia completa de sistemas de software que pueden ser generados. Representa el espacio de posibilidades definido por el conjunto de activos.

2. **Activos Centrales (*Core Assets*):** Son los componentes fundamentales (código, arquitectura, modelos de diseño, documentación, planes de prueba) diseñados específicamente para ser compartidos y reutilizados en toda la línea. La inversión inicial en estos activos es clave para el éxito de la estrategia.
3. **Variabilidad:** Es la propiedad esencial de una SPL. Define la capacidad de la línea para ser adaptada, personalizada o configurada para generar productos específicos que satisfagan las necesidades particulares de un cliente o escenario. La variabilidad es, en esencia, el conjunto de diferencias controladas entre los productos posibles.
4. **Puntos de Variabilidad (*Variability Points*):** Son las ubicaciones explícitas dentro de los Activos Centrales donde se deben tomar decisiones para configurar un producto. Estos puntos actúan como "interruptores" o "opciones" que guían el proceso de personalización.

La motivación detrás de la SPL es fundamentalmente económica y técnica, pues permite el ahorro de costos, ya que al compartir la gran mayoría de los activos (entre el 70% y 90% del sistema), se evita la reimplementación del mismo código y diseño para cada nuevo producto, además, provee de una reducción del tiempo de comercialización (*Time-to-Market*) pues la creación de un nuevo producto se reduce de un ciclo completo de desarrollo a un simple proceso de configuración y ensamblaje de activos preexistentes y permite una mejora de la calidad, pues como los activos centrales, al ser reutilizados y probados en múltiples productos, alcanzan un nivel de madurez y robustez superior al que podría lograrse en un desarrollo de producto único [Ingeniería de Software].

La SPL opera a través de dos procesos interdependientes que deben gestionarse de manera continua, siendo el primero de estos la Inge-

nería del Dominio (*Domain Engineering*), proceso hacia arriba que se enfoca en la creación y el mantenimiento de los Activos Centrales, cuya tarea principal es el modelado de la variabilidad para comprender y documentar todas las posibles configuraciones que la línea puede soportar y que constituye la base teórica y formal de la línea, y un segundo, la Ingeniería de la Aplicación (*Application Engineering*), el cual es el proceso hacia abajo que utiliza los Activos Centrales para configurar y generar productos específicos, que implica tomar las decisiones de variabilidad definidas en el modelo (seleccionar las opciones) para producir una instancia concreta de la línea de productos. La herramienta principal y estándar de facto para formalizar y gestionar la variabilidad durante la Ingeniería del Dominio es el FM. Este modelo establece el puente formal entre las opciones de la línea de productos y la capacidad de generar una instancia válida, lo cual es el principio que se busca trasladar al diseño curricular.

1.1.1.2 Modelos de Características

Es necesario contar con un modelo que represente de manera explícita las variaciones y opciones disponibles en la línea de productos. Este modelo debe ser capaz de capturar la complejidad de las relaciones entre las características y sus variaciones, así como las restricciones que pueden existir entre ellas [Pohl et al., 2005]. La representación gráfica de estas relaciones es fundamental para facilitar la comprensión y el análisis de la variabilidad en la línea de productos.

El Modelo de Características (*Feature Model* - FM) es la técnica de modelado central en la Ingeniería de Líneas de Productos de Software. Su formalismo es indispensable para pasar de la noción abstracta de variabilidad a una representación estructurada y analizable [PTC, 2025]. Los Modelos de Características fueron formalmente introducidos por Kang et al. [1990] como parte de la metodología Análisis de Dominio Orientado a Características (*Feature-Oriented Domain Analy-*

sis) (FODA), un enfoque pionero para la ingeniería de dominios. Desde su concepción, un FM ha sido reconocido como el artefacto primario para la captura de la variabilidad funcional de un sistema. En esencia, un FM es un modelo de dominio que no solo documenta las posibles funcionalidades, sino que también define las reglas de composición entre ellas [GeeksforGeeks].

Formalmente, un Modelo de Características se define como un árbol de decisiones jerárquico donde cada nodo es una característica (*feature*). Una característica se entiende como una unidad de funcionalidad, un requisito, o un aspecto distintivo de un sistema, tal como es percibido por el usuario final o el analista del dominio.

La estructura del FM es un árbol de composición, donde:

- La Característica Raíz representa el sistema o la línea de producto completa (ej., la carrera de Ingeniería Informática).
- Las características descendientes detallan la composición de sus características padre hasta alcanzar las características hoja, que son las unidades más pequeñas y configurables.

Relaciones estructurales

La potencia del FM reside en su notación gráfica concisa, la cual utiliza la posición y símbolos específicos para codificar las restricciones de configuración. Las relaciones entre una característica padre (letra P) y sus hijos (letra C) definen las decisiones de inclusión o exclusión; véase la Tabla 1.1 [Pohl et al., 2005, Kang et al., 1990, Benavides et al., 2010b,a, Wang et al., 2007].

Cuadro 1.1: Notación semántica de las relaciones entre características de los FM.
Fuente: Elaboración propia.

Relación	Semántica Formal	Implicación en la Configuración
Obligatoria	$P \rightarrow C$	Si el padre P es seleccionado, el hijo obligatorio C debe incluirse. No hay variabilidad en este punto
Opcional	$C \rightarrow P$	El hijo C puede ser incluido o excluido, si se selecciona el padre P. Introduce variabilidad
Agrupación AND	$(P \rightarrow \bigwedge_{i=1}^n C_i) \quad \wedge \quad \bigwedge_{i=1}^n (C_i \rightarrow P)$	Si se selecciona la característica padre P, todos los hijos C_i deben incluirse; cada hijo seleccionado implica la presencia del padre. No introduce variabilidad dentro del grupo
Agrupación OR	$(P \rightarrow \bigvee_{i=1}^n C_i) \quad \wedge \quad \bigwedge_{i=1}^n (C_i \rightarrow P)$	Al seleccionar el padre P, debe elegirse al menos un hijo C_i . Cada hijo seleccionado requiere que el padre esté presente. Sí introduce variabilidad

Continúa en la siguiente página

Cuadro 1.1 – Continuación de la página anterior

Relación	Semántica Formal	Implicación en la Configuración
Agrupación alternativa	$(P \rightarrow \bigvee_{i=1}^n C_i) \wedge$ $\bigwedge_{i \neq j} \neg(C_i \wedge C_j) \wedge$ $\bigwedge_{i=1}^n (C_i \rightarrow P)$	Cuando el padre P se selecciona, exactamente un hijo C_i puede activarse y la elección de cualquier hijo obliga a seleccionar al padre. Sí introduce variabilidad

La figura 1.1 muestra los símbolos gráficos para representar las relaciones estructurales.

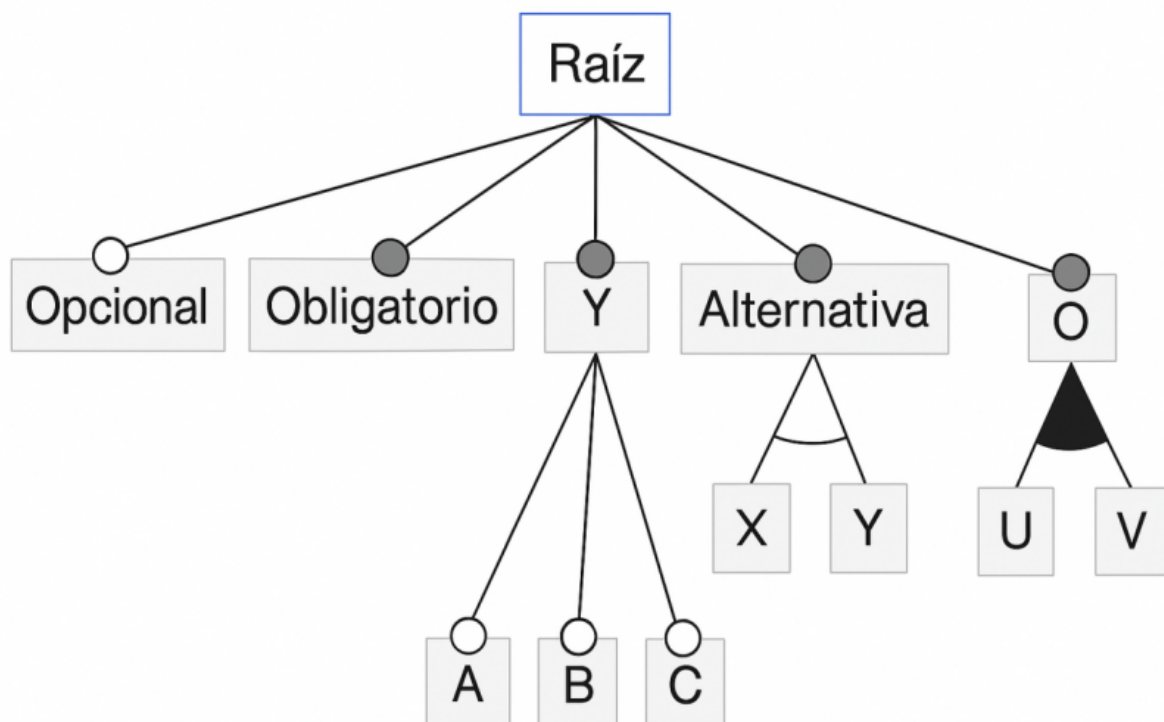


Figura 1.1: Símbolos gráficos para representar las relaciones estructurales. Fuente: Elaboración propia.

Restricciones (*Cross-Tree*)

Las relaciones jerárquicas son insuficientes para capturar todas las reglas de negocio complejas. Por ello, los FM permiten definir las restricciones transversales (*cross-tree*); que son reglas lógicas que definen dependencias entre características que no están directamente conectadas en la jerarquía padre-hijo del árbol de características. Su función es garantizar la validez de una configuración al capturar las relaciones complejas del mundo real que la estructura jerárquica por sí sola no puede expresar [Sundermann et al., 2023, Benavides et al., 2005].

Las dos formas más comunes y fundamentales son:

- **Requiere:** Esta restricción establece una dependencia de presencia. Si una característica A requiere una característica B, entonces siempre que A sea seleccionada en una configuración, B también debe estarlo. Porejemplo, en un modelo de características para un automóvil, la característica "Navegación" puede requerir la característica "Pantalla Táctil". Esta relación se modela en lógica proposicional como el condicional $A \rightarrow B$ [Sundermann et al., 2023, Benavides et al., 2005].
- **Excluye:** Esta restricción establece un conflicto o incompatibilidad. Si una característica C excluye una característica D, entonces ambas no pueden formar parte de la misma configuración. Por ejemplo, el motor "Diésel" puede excluir la transmisión "Eléctrica" en un vehículo híbrido. Su representación lógica es la negación de la conjunción: $\neg(C \wedge D)$ [Sundermann et al., 2023, Benavides et al., 2005].

Semántica Formal y Solucionadores SAT (*SAT Solvers*)

La semántica formal de un FM proporciona un significado matemático preciso a todos sus elementos (jerarquía, relaciones y restricciones), lo que permite un análisis automático y libre de ambigüedades.

- **Traducción a Lógica Proposicional:** La base de la semántica formal es la traducción de todo el FM a una fórmula en lógica proposicional. Cada característica se representa como una variable

proposicional (que puede ser Verdadera o Falsa, indicando si está seleccionada o no). Las relaciones padre-hijo y las restricciones cross-tree se convierten en una serie de fórmulas lógicas que, en conjunto, definen el conjunto de todas las configuraciones válidas [Sundermann et al., 2023, Thüm et al., 2014].

- **Satisfacibilidad (Satisfacibilidad Booleana (*Boolean Satisfiability*) (SAT)) y Conteo de Modelos (*Model Counting*) (SHARPSAT):**

Una vez el FM está representado como una fórmula lógica F , se pueden realizar diversos análisis:

- **Análisis de SAT:** Consiste en verificar si existe al menos una asignación de valores (Verdadero/Falso) a las variables que haga que la fórmula F sea verdadera. Esto responde a la pregunta fundamental de si el FM tiene al menos una configuración válida [Sundermann et al., 2023, Benavides et al., 2010a, Mendonca and Wasowski, 2009].
- **Análisis de SHARPSAT:** Consiste en el conteo de modelos (*Model Counting*) que satisfacen F . Esto es crucial para determinar el número total de configuraciones válidas que define un FM, una métrica esencial para comprender su variabilidad y complejidad [Sundermann et al., 2023, Benavides et al., 2010a, Mendonca and Wasowski, 2009].
- **Solucionadores SAT (*SAT solvers*):** Un solucionador SAT (*SAT solver*) es una herramienta algorítmica altamente optimizada diseñada para resolver problemas de satisfacibilidad. Dada una fórmula F , el solucionador (*solver*) determina si es satisfacible (SAT) o insatisfacible (UNSAT). Para el análisis de SHARPSAT, se emplean solucionadores especializados de conteo de modelos (*#SAT solvers*) como sharpSAT y Ganak, que han demostrado un rendimiento excelente al calcular el número de configuraciones en modelos industriales [Sundermann et al., 2023, Soos et al., 2020].

1.1.1.3 Meta-modelo Educativo

Un Meta-modelo Educativo es un modelo de alto nivel que define los conceptos, las relaciones y las restricciones necesarias para especificar, de manera formal y estandarizada, cómo se deben diseñar aplicaciones de software educativo [Estrada Perea and Giraldo, 2022]. Su propósito fundamental es servir como un andamiaje conceptual que garantiza la inclusión de criterios esenciales —como la usabilidad y el conocimiento pedagógico— que, de otro modo, podrían omitirse en el proceso de desarrollo, dando como resultado herramientas educativas más robustas, eficaces y fáciles de usar [Estrada Perea].

A diferencia de un modelo educativo, que se enfoca en métodos de enseñanza y relaciones en el aula, un meta-modelo opera en un nivel de abstracción superior. Mientras un modelo educativo podría ser, por ejemplo, el Modelo de Transformación Académica (Modelo de Transformación Académica (META)) que se centra en competencias y el trabajo autónomo del alumno [Hernández and Rojas, 2019], un meta-modelo educativo define el lenguaje y las reglas para crear dichos modelos y, crucialmente, para traducirlos a artefactos de software concretos [Estrada Perea and Giraldo, 2022].

La necesidad de un meta-modelo surge de problemas recurrentes en el diseño de software educativo, donde a menudo se carece de:

- Recursos para soportar el trabajo colaborativo.
- Estrategias efectivas para la retroalimentación de las acciones del estudiante.
- Sistemas de ayuda para capacitar a usuarios sin experiencia [Estrada Perea].

Al definir formalmente estos elementos y sus interrelaciones, tu meta-modelo se convierte en la columna vertebral de tu servidor (*back-end*). Este permite que el sistema no solo almacene información, sino

que comprenda y procese la lógica fundamental de la variabilidad curricular, asegurando que todas las configuraciones generadas sean pedagógicamente sólidas y estructuralmente consistentes.

La literatura plantea que se puede estructurar un meta-modelo alrededor de los siguientes componentes [Estrada Perea and Giraldo, 2022]:

- **Conceptos:** Las entidades fundamentales (asignatura, competencia, modalidad, estilo de aprendizaje).
- **Atributos:** Las propiedades que caracterizan a cada concepto (nombre, créditos, obligatoriedad).
- **Relaciones Estructurales y Dinámicas:** Cómo se vinculan los conceptos entre sí (prerrequisito de, parte de, excluye a).
- **Restricciones:** Reglas de integridad que garantizan la validez del modelo (ejemplo: "una asignatura optativa no puede ser prerrequisito de más de tres asignaturas obligatorias").

1.1.1.4 Gestión de la variabilidad

La variabilidad es un principio fundamental en los procesos de aprendizaje, que postula que las diferencias individuales entre los estudiantes no son excepciones, sino la regla en cualquier entorno educativo [Rose and Strangman, 2007]. Este concepto rompe con el paradigma de un "alumno promedio" y reconoce que cada estudiante es una combinación única de fortalezas, desafíos e intereses que están interconectados en todo su ser [CAST, 2018].

Desde una perspectiva cognitiva, la variabilidad no es un obstáculo, sino un recurso de aprendizaje esencial. Según la revisión de más de 150 estudios analizada por Raviv et al. [2022], la exposición a estímulos variables conduce a una mejor capacidad para generalizar el conocimiento. Este principio se manifiesta en diferentes dimensiones del aprendizaje:

- En el aprendizaje conceptual: Para que un estudiante comprenda verdaderamente una categoría (como "mamífero"), es crucial que se enfrente a ejemplos diversos y heterogéneos (diferentes tipos de mamíferos), lo que le permite identificar las características relevantes y esenciales, filtrando las irrelevantes [Raviv et al., 2022].
- En el desarrollo de habilidades: La Hipótesis de la Variabilidad, originada de la Teoría del Esquema Motor de Schmidt (1975), postula que una práctica abundante y variada enriquece los esquemas motores generales. Esto le otorga a la persona una amplitud de respuesta para adaptarse y resolver problemas en diferentes escenarios y contextos [Schmidt, 1975].

La implicación pedagógica central es que un enfoque educativo uniforme, que no considera esta diversidad intrínseca, inevitablemente desatiende a la mayoría de los estudiantes [Rose and Strangman, 2007]. Abrazar la variabilidad significa, por tanto, crear experiencias de aprendizaje que sean personales y significativas para esta diversidad de perfiles [CAST, 2018].

La gestión de la variabilidad es el conjunto de procesos, estrategias y acciones deliberadas diseñadas para optimizar el funcionamiento de las instituciones educativas, planificando, organizando, dirigiendo y controlando los recursos con el objetivo explícito de responder a la diversidad del alumnado y alcanzar metas pedagógicas de calidad [Weinstein, 2002].

Esta gestión no es una tarea única, sino una función que se despliega en varias dimensiones interdependientes [Weinstein, 2002]:

- Gestión Pedagógica: Se enfoca en el núcleo del proceso de enseñanza-aprendizaje. Implica diseñar planes y programas académicos que se ajusten a las necesidades y características de los estudiantes, fomentando la innovación en metodologías y la implementación de evaluaciones formativas.

- **Gestión Administrativa:** Asegura una administración eficiente de los recursos humanos, materiales y financieros, creando las condiciones operativas y de infraestructura necesarias para un entorno propicio para el desarrollo de todos los estudiantes.
- **Gestión Comunitaria:** Busca fortalecer los lazos entre la institución educativa, las familias y el entorno social, fomentando un ambiente inclusivo y de convivencia positiva que es fundamental para el aprendizaje.

Para que esta gestión sea efectiva, la literatura especializada identifica factores críticos. Según Weinstein [2002], entre ellos se encuentran un liderazgo legitimado, un sentido de misión compartido, un clima de convivencia positivo y procesos de planificación participativa. La ausencia de un liderazgo efectivo y de herramientas modernas de gestión se cuenta entre los factores que afectan negativamente los resultados de las organizaciones educativas [Weinstein, 2002].

1.1.1.5 Ingeniería de Software Educativo

La Ingeniería de Software es una disciplina que aplica principios y métodos sistemáticos para el desarrollo, operación y mantenimiento de software de calidad [Sommerville, 2016]. A diferencia de la programación individual, esta disciplina se enfoca en la aplicación de un enfoque de ingeniería para construir software confiable y eficiente que satisfaga los requisitos del usuario, considerando restricciones de tiempo y presupuesto [Pressman, 2014].

La ingeniería de software establece un marco metodológico que abarca todo el ciclo de vida del desarrollo, desde la captura de requisitos hasta el mantenimiento del producto final, garantizando la calidad mediante procesos estandarizados y mejores prácticas validadas [IEEE, 2014].

El Software Educativo se define como cualquier programa computacional cuyo propósito principal es apoyar el proceso de enseñanza-

aprendizaje, facilitando la adquisición de conocimientos o desarrollo de habilidades específicas [Bates, 2019]. Este tipo de software se caracteriza por su intencionalidad pedagógica explícita, distinguiéndose del software de propósito general por su diseño centrado en objetivos educativos concretos [Bates, 2019].

La efectividad del software educativo depende críticamente de la integración de principios instruccionales sólidos en su diseño, asegurando que no solo sea tecnológicamente robusto, sino también pedagógicamente sólido [Mishra and Koehler, 2006]. La investigación ha demostrado que el software educativo bien diseñado puede mejorar significativamente los resultados de aprendizaje cuando se alinea con estrategias pedagógicas apropiadas [Hattie, 2009].

La Ingeniería de Software Educativo emerge como la intersección disciplinaria que aplica los principios de la ingeniería de software al dominio específico del desarrollo de software educativo [Sommerville, 2016, Bates, 2019]. Esta integración es crucial porque combina el rigor técnico de la ingeniería de software con la fundamentación pedagógica necesaria para crear herramientas educativas efectivas [Mishra and Koehler, 2006].

El desafío central en esta disciplina híbrida reside en equilibrar los requisitos técnicos (rendimiento, confiabilidad, mantenibilidad) con los requisitos pedagógicos (valor educativo, adecuación al nivel del estudiante, alineamiento con objetivos de aprendizaje) [Bates, 2019]. El modelo TPACK (Conocimiento Tecnológico Pedagógico del Contenido (*Technological Pedagogical Content Knowledge*) (TPACK)) enfatiza la necesidad de esta integración, argumentando que la tecnología educativa efectiva requiere una comprensión profunda de cómo la tecnología, la pedagogía y el contenido se interrelacionan [Mishra and Koehler, 2006].

En el contexto de la investigación, la Ingeniería de Software Educativo proporciona el marco metodológico para desarrollar un servidor (*backend*) que no solo sea técnicamente robusto, sino también pedagógi-

camente fundamentado, capaz de gestionar la variabilidad curricular de manera que respete los principios de diseño instruccional y promueva experiencias de aprendizaje significativas.

1.1.1.6 Configurador curricular

En el contexto de las Líneas de Productos de Software (SPL), la configuración se refiere al proceso sistemático de seleccionar un conjunto específico de características de un Modelo de Características para derivar un producto individual que satisfaga requisitos particulares [Apel et al., 2013]. Este proceso implica tomar decisiones de configuración que respeten todas las restricciones definidas en el modelo, asegurando que la selección final de características sea válida y consistente [Benavides et al., 2010a].

La configuración en SPL se caracteriza por ser un proceso guiado por restricciones donde las dependencias entre características (mandatorias, opcionales, alternativas) y las restricciones cross-tree (requiere, excluye) determinan el espacio de configuraciones posibles [Chen et al., 2020]. La complejidad de este proceso aumenta exponencialmente con el tamaño del modelo, haciendo necesario el uso de herramientas automatizadas para garantizar la corrección de las configuraciones generadas [Benavides et al., 2010a].

La planificación curricular es el proceso de diseño y organización sistemática de los elementos que constituyen un plan de estudios, incluyendo objetivos de aprendizaje, contenidos, actividades educativas y criterios de evaluación [UNESCO, 2022]. Tradicionalmente, este proceso ha sido realizado de manera manual por comités académicos, utilizando documentos estáticos y hojas de cálculo que dificultan la gestión de la complejidad y variabilidad inherentes a los programas educativos modernos [Sánchez et al., 2020].

Los desafíos en la planificación curricular contemporánea incluyen la necesidad de crear trayectorias formativas personalizadas que re-

spondan a diferentes perfiles estudiantiles y demandas del mercado laboral, manteniendo al mismo tiempo la consistencia académica y el cumplimiento de los estándares de calidad educativa [Rodríguez et al., 2019]. La gestión manual de estas complejidades resulta en procesos lentos, propensos a errores y difíciles de mantener [Sánchez et al., 2020].

Un Configurador Curricular es una herramienta especializada que aplica los principios de configuración de SPL al dominio de la planificación educativa, permitiendo la generación semi-automatizada de planes de estudio personalizados a partir de un modelo base que captura la variabilidad curricular [Sánchez et al., 2020]. Este sistema opera como un motor de decisiones académicas que guía al usuario (gestor académico, coordinador de programa) a través del espacio de configuraciones válidas, garantizando que el plan de estudio resultante cumpla con todas las restricciones pedagógicas y administrativas [Chen et al., 2020].

El valor fundamental de un configurador curricular reside en su capacidad para reducir la complejidad del diseño curricular mediante la aplicación de restricciones formales que previenen configuraciones inválidas [Rodríguez et al., 2019]. Por ejemplo, el sistema puede asegurar automáticamente que:

- Se cumplan todos los prerrequisitos entre asignaturas
- Se respeten los límites de créditos por semestre
- Se mantenga la coherencia entre las competencias a desarrollar y las asignaturas seleccionadas
- Se eviten conflictos de horario y sobrecarga académica

En el contexto de la investigación, el desarrollo del servidor (*back-end*) para un configurador curricular representa la materialización técnica de este concepto, proporcionando la lógica de negocio, el motor de

reglas y los servicios necesarios para soportar el proceso de configuración curricular de manera robusta, escalable y consistente.

1.2. Análisis del mercado

El presente epígrafe desarrolla un análisis sistemático del mercado relacionado con los sistemas de gestión curricular y herramientas de modelado de variabilidad. Se examinan tanto las soluciones tradicionales empleadas por instituciones académicas como aquellas provenientes del ámbito de la ingeniería de líneas de productos, identificando sus fortalezas, limitaciones y brechas funcionales. A partir de esta revisión, se fundamenta la oportunidad tecnológica que sustenta la propuesta investigativa, destacando el vacío existente entre las plataformas de administración académica y los mecanismos de planificación curricular basados en variabilidad, lo que justifica la necesidad y pertinencia de la solución desarrollada.

1.2.1 Panorama General del Mercado

El mercado de sistemas de gestión curricular presenta una segmentación clara entre soluciones genéricas de administración académica y herramientas especializadas en modelado de variabilidad. Mientras plataformas como Moodle, Blackboard y Canvas dominan el segmento de los Sistemas de Gestión del Aprendizaje (*Learning Management System* - LMS) [Moodle Project, 2025, Anthology Inc., 2025, Instructure, 2025], existen vacíos significativos en soluciones específicas para la planificación curricular basada en Líneas de Productos Educativos [Sánchez et al., 2020].

1.2.2 Análisis de Soluciones Existentes

El análisis de soluciones existentes se organiza en dos grupos clara-

mente diferenciados, cada uno representativo de perspectivas y alcances funcionales opuestos dentro del dominio investigado. Por un lado, se examinan los sistemas institucionales de gestión curricular tradicional, ampliamente implementados en universidades, cuyo propósito principal es administrar información académica pero que presentan limitaciones para modelar y validar variabilidad curricular. Por otro lado, se estudian herramientas avanzadas de ingeniería de líneas de productos de software (SPL), diseñadas para gestionar características y restricciones complejas, pero que requieren conocimientos técnicos elevados y carecen de adaptación al contexto educativo. Esta comparación permite identificar brechas tecnológicas y conceptuales que fundamentan la necesidad de una solución especializada que integre fortalezas de ambos mundos mientras atiende las demandas propias de la planificación curricular.

Sistemas de Gestión Curricular Tradicionales:

- SIGA y Banner: Los sistemas como Banner son fundamentalmente sistemas de información transaccionales diseñados para la gestión operativa de una institución. Su arquitectura está optimizada para manejar grandes volúmenes de datos de estudiantes, matrículas, registros de cursos y pagos. La evidencia de su funcionamiento, como se observa en la documentación de "Banner Student", muestra una estructura basada en bloques, ventanas y secciones para la gestión de admisiones, currículums y campos de estudio (Ellucian Company, 2015b) [Ellucian Company, 2015]. Esta lógica está orientada a capturar y recuperar información de forma consistente, no a modelar relaciones complejas y variables entre componentes curriculares. Carecen de un motor de reglas nativo que permite definir y verificar restricciones complejas como las que se modelan con "requiere" o "excluye" en un Modelo de Características (*Feature Model*). Su limitación principal es que operan sobre una estructura de datos fija y predefinida, lo que los hace rígidos para experimen-

tar con diferentes configuraciones curriculares o para gestionar una línea de productos educativos donde las relaciones entre asignaturas y competencias son dinámicas y variables.

- Moodle, como Sistema de Gestión del Aprendizaje (*Learning Management System*) [Moodle Project, 2025], es una plataforma excelente para la ejecución y gestión de un plan de estudios ya definido. Su fortaleza reside en organizar cursos, desglosarlos en unidades y lecciones, gestionar la entrega de contenidos, las actividades de evaluación y la interacción entre estudiantes y profesores [Moodle Project, 2025]. Sin embargo, su funcionalidad no se extiende al diseño instruccional ni a la planificación curricular basada en características. Moodle asume que el diseño del curso (los objetivos, la secuencia de contenidos, las actividades de evaluación) ya ha sido realizado externamente. No proporciona herramientas para que los gestores académicos modelen visualmente un plan de estudios, definan características opcionales u obligatorias, o establezcan restricciones entre ellas. En el contexto de tu investigación, Moodle sería un consumidor potencial de la configuración curricular final generada por tu servidor de aplicaciones (*backend*), pero no una herramienta para crearla.

Herramientas de Modelado de Variabilidad:

- `pure::variants`: es una solución comercial robusta para la ingeniería de líneas de productos de software (SPL). Proporciona un entorno completo para definir características, crear modelos, gestionar restricciones y, crucialmente, derivar productos concretos a partir de configuraciones válidas. Su potencia es innegable en el dominio técnico para el que fue diseñado [pure-systems GmbH, 2024]. El problema central para tu contexto educativo es su alta barrera de entrada para usuarios no técnicos. Los gestores académicos y diseñadores curriculares, que son expertos en pedagogía pero no

necesariamente en SPL, se verían enfrentados a una terminología y flujos de trabajo propios de la ingeniería de software. Adaptar pure::variants requeriría una personalización significativa de la interfaz y una capacitación intensiva, lo que dificulta su adopción ágil en un entorno académico, validando la necesidad de una solución más especializada y accesible.

- FeatureIDE: es un entorno de código abierto construido como un complemento (*plugin*) para Eclipse, lo que lo hace muy popular en entornos de investigación y enseñanza de la ingeniería de software [Thüm et al., 2014]. Al igual que pure::variants, es poderoso para el análisis y la configuración de modelos de características. Sin embargo, su dependencia de Eclipse y su orientación hacia desarrolladores lo convierten en una herramienta que requiere especialización técnica. Para un coordinador de carrera o un vicerrector académico, interactuar con FeatureIDE supondría una curva de aprendizaje muy pronunciada. Su arquitectura no está diseñada para integrarse de manera natural con los sistemas de información universitarios existentes (como los propios SIGA o Banner), lo que limita su utilidad práctica para la gestión curricular del día a día. Su uso en educación se limita predominantemente a la enseñanza de conceptos de SPL, no a la gestión operativa de currículos.

1.2.3 Oportunidades de Mercado Identificadas

Del análisis realizado se evidencia que el mercado asociado a la temática padece de un vacío crítico en herramientas capaces de integrar la gestión de variabilidad, propia de las SPL, con la planificación curricular educativa, creando una oportunidad para sistemas especializados [Sánchez et al., 2020]. Las herramientas existentes de SPL están diseñadas para ingenieros de software, no para profesionales de la educación, limitando su adopción en instituciones educativas [Chen et al., 2020].

En este escenario, la necesidad de proponer una solución propia no surge de la ausencia total de tecnologías, sino de la falta de una alternativa que articule las fortalezas identificadas en ambos tipos de herramientas y las adapte al dominio educativo. De los sistemas analizados se reconoce como indispensable la capacidad de representar explícitamente características con relaciones jerárquicas y restricciones; la validación automática de configuraciones; la posibilidad de derivar múltiples variantes a partir de un modelo común; y la interoperabilidad con ecosistemas académicos existentes. Sin embargo, también se observa que estas capacidades requieren una interfaz y flujo conceptual comprensible para gestores curriculares y especialistas pedagógicos, no solamente para ingenieros de software.

La solución propuesta se posiciona estratégicamente en el mercado educativo como un "Motor de Variabilidad Curricular" especializado, que capitaliza las tendencias actuales de crecimiento en educación híbrida y multimodal, así como las demandas crecientes de personalización educativa y eficiencia en la gestión académica post-pandemia. Su propuesta de valor se sustenta en fortalezas técnicas diferenciadoras, ofreciendo un servidor de aplicaciones (*backend*) que permita diseñar y gestionar modelos de características aplicados a planes de estudio, y derivar itinerarios formativos dinámicos y personalizables. Su valor diferencial radica en su especialización en gestión de variabilidad curricular, incorporar motores de reglas pedagógicas que comprenden prerrequisitos, competencias y secuencias, una arquitectura de Interfaz de Programación de Aplicaciones (*Application Programming Interface - API*) para integración con sistemas existentes y un modelo de datos optimizado para planificación académica. Estas capacidades se traducen en ventajas competitivas tangibles como la automatización de validaciones que reduce errores en diseño curricular, la generación de múltiples variantes de planes de estudio desde un tronco común y su adaptabilidad a contextos institucionales diversos, posicionándose como com-

plemento esencial -no competencia- de los sistemas LMS existentes, al enfocarse específicamente en el diseño y planificación académica más que en la ejecución curricular. En consecuencia, la propuesta investigativa no se limita a replicar herramientas existentes, sino que se posiciona como un puente necesario entre el rigor formal de las SPL y la flexibilidad curricular demandada por la educación contemporánea, constituyendo así una innovación con pertinencia tecnológica y académico-formativa.

1.3. Metodología de desarrollo del software

Las metodologías de desarrollo de software son enfoques estructurados que guían las distintas fases del ciclo de vida de un proyecto, desde la planificación y análisis hasta la implementación y mantenimiento. Estas metodologías establecen principios, prácticas y procedimientos para gestionar el desarrollo de manera organizada y eficiente, promoviendo la colaboración en equipo y la calidad del producto final [Pacienza, 2015].

A lo largo de esta sección se realiza un análisis de las metodologías de software disponibles y, finalmente, se detallan los principios, fases y prácticas de la metodología seleccionada para guiar el diseño e implementación del sistema propuesto. La elección de la metodología es fundamental para asegurar un proceso de desarrollo ágil, eficiente y adaptable a posibles cambios en los requisitos del proyecto. Estas metodologías facilitan además la detección temprana de errores y el ajuste progresivo de funcionalidades, garantizando la calidad final del sistema.

En la actualidad se pueden diferenciar dos grandes grupos de metodologías de desarrollo de software: las ágiles y las tradicionales. A continuación, se explican las características de cada una de ellas.

1.3.1 Metodologías de desarrollo de software tradicionales

Las metodologías de desarrollo de software tradicionales se caracterizan por definir total y rígidamente los requisitos al inicio de los proyectos de ingeniería de software. Los ciclos de desarrollo son poco flexibles y no permiten realizar cambios, al contrario que las metodologías ágiles; lo que ha propiciado el incremento en el uso de las segundas.

La organización del trabajo de las metodologías tradicionales es lineal, es decir, las etapas se suceden una tras otra y no se puede empezar la siguiente sin terminar la anterior. Tampoco se puede volver hacia atrás una vez se ha cambiado de etapa. Estas metodologías, no se adaptan nada bien a los cambios, y el mundo actual cambia constantemente [Awad, 2005]. Las principales metodologías tradicionales o clásicas son:

- **Modelo en Cascada:** también conocido como el ciclo de vida clásico del software, es una de las metodologías de desarrollo de software más antiguas y ampliamente utilizadas. Este enfoque se basa en la idea de que el desarrollo de software debe seguir una secuencia lineal y predefinida de etapas de riguroso orden, cada una de las cuales debe completarse antes de pasar a la siguiente. Los requisitos y especificaciones iniciales no están predispuestos para cambiarse.
- **Incremental:** en esta metodología de desarrollo de software se va construyendo el producto final de manera progresiva. En cada etapa incremental se agrega una nueva funcionalidad, lo que permite ver resultados de una forma más rápida en comparación con el modelo en cascada. El software se puede empezar a utilizar incluso antes de que se complete totalmente y, en general, es mucho más flexible que las demás metodologías.
- **Espiral:** es una combinación de los dos modelos anteriores, que

añade el concepto de análisis de riesgo. Se divide en cuatro etapas: planificación, análisis de riesgo, desarrollo de prototipo y evaluación del cliente. El nombre de esta metodología da nombre a su funcionamiento, ya que se van procesando las etapas en forma de espiral. Cuanto más cerca del centro se está, más avanzado está el proyecto.

1.3.2 Metodologías de desarrollo de software ágiles

Hoy en día las metodologías ágiles de desarrollo de software son las más utilizadas debido a su alta flexibilidad y agilidad. Los equipos de trabajo que las utilizan son mucho más productivos y eficientes, ya que saben lo que tienen que hacer en cada momento. Además, la metodología permite adaptar el software a las necesidades que van surgiendo por el camino, lo que facilita construir aplicaciones más funcionales.

Las metodologías ágiles se basan en la metodología incremental, en la que en cada ciclo de desarrollo se van agregando nuevas funcionalidades a la aplicación final. Sin embargo, los ciclos son mucho más cortos y rápidos, por lo que se van agregando pequeñas funcionalidades en lugar de grandes cambios.

Este tipo de metodologías permite construir equipos de trabajo autosuficientes e independientes que se reúnen cada poco tiempo para poner en común las novedades. Poco a poco, se va construyendo y puliendo el producto final, a la vez que el cliente puede ir aportando nuevos requerimientos o correcciones, ya que puede comprobar cómo avanza el proyecto en tiempo real [Pacienza, 2015]. Las principales metodologías ágiles son:

- Kanban: metodología de trabajo inventada por la empresa de automóviles Toyota. Consiste en dividir las tareas en porciones mínimas y organizarlas en un tablero de trabajo dividido en tareas pendientes, en curso y finalizadas. De esta forma, se crea un flujo de

trabajo muy visual basado en tareas prioritarias e incrementando el valor del producto.

- Scrum: es también una metodología incremental que divide los requisitos y tareas de forma similar a Kanban. Se itera sobre bloques de tiempos cortos y fijos (entre dos y cuatro semanas) para conseguir un resultado completo en cada iteración. Las etapas son: planificación de la iteración (*planning sprint*), ejecución (*sprint*), reunión diaria (*daily meeting*) y demostración de resultados (*sprint review*). Cada iteración por estas etapas se denomina también *sprint*.
- Extreme Programming (XP): es una metodología de desarrollo de software basada en las relaciones interpersonales, que se consideran la clave del éxito. Su principal objetivo es crear un buen ambiente de trabajo en equipo y que haya un *feedback* constante del cliente. El trabajo se basa en 12 conceptos: diseño sencillo, *testing*, refactorización y codificación con estándares, propiedad colectiva del código, programación en parejas, integración continua, entregas semanales e integridad con el cliente, *cliente in situ*, entregas frecuentes y planificación.

1.3.3 Elección de la metodología

En las metodologías tradicionales el principal problema es que nunca se logra planificar bien el esfuerzo requerido para seguir la metodología. Pero entonces, si se logran definir métricas que apoyen la estimación de las actividades de desarrollo, muchas prácticas de metodologías tradicionales podrían ser apropiadas. El no poder predecir siempre los resultados de cada proceso no significa que se esté frente a una disciplina de azar. Lo que significa es que se está frente a la necesidad de adaptación de los procesos de desarrollo que son llevados por parte de los equipos que desarrollan software [Awad, 2005].

Tener metodologías diferentes para aplicar de acuerdo con el proyec-

to que se desarrolle resulta una idea interesante. Estas metodologías pueden involucrar prácticas tanto de metodologías ágiles como de metodologías tradicionales. De esta manera se podría tener una metodología por cada proyecto, la problemática sería definir cada una de las prácticas, y en el momento preciso definir parámetros para saber cuál usar. Es importante tener en cuenta que el uso de un método ágil no vale para cualquier proyecto. Sin embargo, una de las principales ventajas de los métodos ágiles es su peso inicialmente ligero y por eso las personas que no estén acostumbradas a seguir procesos encuentran estas metodologías bastante agradables.

La tabla 1.2 explica de forma resumida las diferencias que existen entre las metodologías ágiles y las tradicionales:

Cuadro 1.2: Comparación, Metodología Ágil VS Metodología Tradicional [Awad, 2005]

Parámetros	Metodologías Ágiles	Metodologías Tradicionales
Acercamiento	Adaptivo	Predictivo
Medición del objetivo	Valor para el negocio	Conformación del plan
Tamaño del proyecto	Pequeño	Largo
Tipo de administración	Descentralizado	Autocrático
Perspectiva de cambios	Adaptable a cambio	Sostenible al cambio
Cultura del equipo	Liderazgo\ Colaboración	Comandada\ Controlada
Documentación	Baja	Pesada
Orientada	Personas	Procesos
Ciclos de desarrollo	Numerosos	Limitados
Dominio de desarrollo	Impredecibles	Predecibles
Planificación inicial	Mínima	Comprehensiva

Parámetros	Metodologías Ágiles	Metodologías Tradicionales
Retorno de la inversión	Temprano en el proyecto	Al final del proyecto
Tamaño del equipo	Pequeño	Grande

Para la selección de la metodología a utilizar se toma como referencia la tabla 1.2. Para ello se analizan las características del proyecto atendiendo a los diferentes aspectos y se selecciona la metodología con la que haya más coincidencias, dando como resultado de este análisis la tabla 1.3.

Cuadro 1.3: Comparación de la Metodología Ágil y la Metodología Tradicional con respecto al proyecto. Fuente: Elaboración propia

Parámetros	Ágiles	Pesadas
Acercamiento	X	
Medición del objetivo	X	
Tamaño del proyecto	X	
Tipo de administración	X	
Perspectiva de cambios	X	
Cultura del equipo	X	
Documentación	X	
Orientada	X	
Ciclos de desarrollo	X	
Dominio de desarrollo	X	
Planificación inicial	X	
Retorno de la inversión		X
Tamaño del equipo	X	
Total:	12	1

Como resultado se obtiene que lo mejor es utilizar una metodología ágil. Ya que, según el análisis realizado, se acomoda mejor a las necesi-

dades y características del proceso a llevar a cabo. Para decidir qué metodología ágil se utilizará, se realizó una comparación entre algunas de las metodologías ágiles más comúnmente utilizadas, la cual se refleja en la tabla 1.4.

Cuadro 1.4: Comparación de Metodología Ágiles. Fuente: Elaboración propia

Parámetros	XP	Scrum
Duración	1 a 2 semanas	2 semanas a 1 mes
Cambios	Sensibles al cambio	No permite cambios
Prioridad	Estricta, priorizada por el cliente	Determinada por el equipo
Prácticas de ingeniería	Prescribe prácticas (pruebas, programación en parejas)	No prescribe prácticas
Tamaño de los proyectos	Pequeños y medianos	Pequeños, medianos y grandes
Tamaño de equipo	Menor que 10	Múltiples equipos menores que 10

Teniendo en cuenta la tabla 1.4 se decide utilizar la metodología XP. Esto se debe a que es muy eficiente durante el proceso de pruebas y planificación, su tasa de error es muy pequeña, facilita los cambios, origina una programación muy organizada, además de fomentar la comunicación entre los desarrolladores y los clientes. También se puede aplicar a cualquier lenguaje de programación, el cliente tiene control sobre las prioridades, durante todo el proyecto se pueden realizar diversas pruebas y, sobre todo, permite ahorrar mucho tiempo y dinero.

1.3.4 Programación Extrema

XP es una metodología ágil de desarrollo de software con bases en la comunicación constante y la retroalimentación. Uno de sus fines principales es el de construir un producto que vaya en línea con los re-

querimientos del cliente. En ese sentido es adaptable a los cambios, generando una rápida respuesta frente a cualquier inconveniente. Por otro lado, el equipo de trabajo tiene la ventaja de potenciar sus relaciones, ya que el proceso que de este se desprende es abierto, conjunto y de aprendizaje continuo [Pressman, 2014].

Las 4 etapas que sigue XP son:

- Planificación: toma como referencia la identificación de las Historias de Usuario (HU) con pequeñas versiones que se irán revisando en periodos cortos con el fin de obtener un software funcional.
- Diseño: trabaja el código orientado a objetivos y, sobre todo, usando los recursos necesarios para que funcione.
- Codificación: se refiere al proceso de programación organizada en parejas, estandarizada y que resulte en un código universal entendible.
- Pruebas: consiste en un testeo automático y continuo en el que el cliente tiene voz para validar y proponer.

Los principios de XP comprenden diez buenas prácticas que involucran al equipo de trabajo, los procesos y el cliente [Pressman, 2014]; los cuales son:

- Planificación incremental: Se toman los requerimientos en HU, las cuales son negociadas progresivamente con el cliente.
- Entregas pequeñas: Se desarrolla primero la más mínima parte útil que le proporcione funcionalidad al sistema, y poco a poco se efectúan incrementos que añaden funcionalidad a la primera entrega, cada ciclo termina con una entrega del sistema.
- Diseño sencillo: Solo se efectúa el diseño necesario para cumplir con los requerimientos actuales, es decir, no se abordan requerimientos futuros.

- Desarrollo previamente aprobado: Una de las características relevantes y propias de XP es que primero se escriben las pruebas y luego se da la codificación, esto con la finalidad de asegurar la satisfacción del requerimiento.
- Limpieza del código o refactorización: Consiste en simplificar y optimizar el programa sin perder funcionalidad, es decir, alterar su estructura interna sin afectar su comportamiento externo.
- Programación en parejas: Es otra de las características de esta metodología, que propone que los desarrolladores trabajen en parejas en una terminal, verificando cada uno el trabajo del otro y ayudándose para buscar las mejores soluciones. Se entiende que de esta forma el trabajo será más eficiente y de mayor calidad.
- Propiedad colectiva: El conocimiento y la información deben ser de todos, por lo tanto no se desarrollan islas de conocimiento, todos los programadores poseen todo el código y cualquiera puede sugerir y realizar mejoras.
- Integración continua: Al terminar una tarea, esta se integra al sistema entero y se realizan pruebas de unidad a todo el sistema, esta práctica permite que la aplicación sea más funcional en cada iteración y garantiza su funcionamiento con los demás módulos del sistema.
- Ritmo sostenible: No es aceptable trabajar durante grandes cantidades de horas ya que se considera que puede reducir la calidad del código y la productividad del equipo a mediano plazo, se sugieren 40 horas semanales.
- Cliente presente: Se debe tener un representante (cliente o usuario final) a tiempo completo, ya que en XP este hace parte del equipo de desarrollo y es responsable de formular los requerimientos para el desarrollo del sistema.

1.3.4.1 Artefactos

Las metodologías utilizan artefactos para describir las actividades o procedimientos que se deben seguir durante su desarrollo. A continuación, se describen los artefactos de XP:

- **HU:** Representan una breve descripción del comportamiento del sistema. Emplea terminología del cliente sin lenguaje técnico. Se realiza una por cada característica principal del sistema. Se emplean para hacer estimaciones de tiempo y para el plan de lanzamientos. Reemplazan un gran documento de requisitos y presiden la creación de las pruebas de aceptación. Estas deben proporcionar sólo el detalle suficiente como para poder hacer razonable la estimación de cuánto tiempo requiere la implementación de la HU, difiere de los casos de uso porque son escritos por el cliente, no por los programadores, empleando terminología del cliente [Pressman, 2014].
- **Tareas de Ingeniería:** Una HU se descompone en varias tareas de ingeniería, que describen las actividades que se realizarán en cada una de ellas. Así mismo, las tareas de ingeniería se vinculan más al desarrollador, ya que permite tener un acercamiento con el código [Pressman, 2014].
- **Tarjetas Clase, Responsabilidad y Colaboración (CRC):** Estas tarjetas se dividen en tres secciones que contienen la información del nombre de la clase, sus responsabilidades y sus colaboradores. Una clase es cualquier persona, cosa, evento, concepto, pantalla o reporte. Las responsabilidades de una clase son las cosas que conoce y las que realiza, sus atributos y métodos. Los colaboradores son las demás clases con las que trabaja en conjunto para llevar a cabo sus responsabilidades [Pressman, 2014].

En síntesis, actualmente XP es una de las metodologías de mayor

aceptación en la industria del software. Su enfoque basado en los métodos ágiles, su énfasis en la gestión del recurso humano, el cual es uno de los puntos más críticos en todo proyecto, y sus principios de previsibilidad y adaptabilidad hacen de esta metodología una buena opción a seguir.

1.4. Herramientas y tecnologías

La implementación de la solución propuesta requiere la selección cuidadosa de tecnologías, herramientas y *frameworks* que permitan materializar de manera eficiente los modelos curriculares, soportar reglas de variabilidad y garantizar una experiencia de desarrollo fluida. En este epígrafe se describen los componentes tecnológicos que conformarán la plataforma, justificando su elección en función de Solo FastAPIs como escalabilidad, mantenibilidad, interoperabilidad y robustez. Asimismo, se analizan los entornos de desarrollo, lenguajes de programación, sistemas de persistencia y bibliotecas especializadas que facilitan la representación formal de restricciones curriculares y la validación automática de configuraciones. Esta caracterización permite comprender la base tecnológica que sustenta la solución y cómo cada herramienta contribuye al logro de sus objetivos funcionales y no funcionales.

1.4.1 Lenguaje de Programación

Los lenguajes de programación son herramientas formales que permiten expresar instrucciones de manera estructurada para que un sistema computacional ejecute tareas determinadas [Sebesta, 2016]. A través de ellos se implementan algoritmos, modelos de datos, reglas de negocio y mecanismos de interacción con el usuario u otros sistemas. La elección de un lenguaje influye directamente en la escalabilidad, el rendimiento, la mantenibilidad y la capacidad de expresar soluciones complejas de manera eficiente. En el contexto del sistema propuesto,

el lenguaje seleccionado debe ofrecer flexibilidad, capacidad de representación estructural y soporte robusto para lógica y concurrencia [Somerville, 2016].

Python será el lenguaje principal utilizado para el desarrollo del *backend* de la solución propuesta. Su elección se fundamenta en su madurez tecnológica, su amplio ecosistema de librerías y la claridad sintáctica que facilita la implementación de modelos complejos manteniendo niveles altos de legibilidad y mantenibilidad del código [Van Rossum and Drake, 1995, Lutz, 2013]. Python es ampliamente utilizado en el ámbito académico y profesional, lo que garantiza disponibilidad de documentación, ejemplos de buenas prácticas y una sólida comunidad de soporte [Ray et al., 2014].

Una de las principales fortalezas de Python radica en su capacidad de integración con múltiples paradigmas de desarrollo, permitiendo tanto programación estructurada como orientada a objetos y funcional [Lutz, 2013]. Esto resulta particularmente útil para el diseño modular del sistema, la definición formal de FM y la implementación de validaciones de reglas en motores lógicos.

La representación estructurada de modelos de características FM requiere un lenguaje capaz de expresar jerarquías, relaciones lógicas, restricciones transversales y arinalidades con claridad y sin ambigüedad. Python ofrece capacidades idóneas para este propósito debido a su flexibilidad en estructuras de datos y su riqueza semántica para definir invariantes y reglas de negocio.

En primer lugar, Python permite modelar FM de forma natural mediante diccionarios, listas y clases, manteniendo la semántica jerárquica del dominio sin necesidad de sintaxis complejas ni compilación intermedia. Esto facilita representar nodos, ramas, grupos OR/XOR, atributos y relaciones cross-tree como estructuras anidadas. Gracias a esta expresividad, resulta posible describir distintos niveles del modelo —desde características obligatorias y opcionales hasta restricciones de cardinal-

idad— de manera directa y legible, lo que favorece tanto su mantenimiento como su inspección.

Además, Python ofrece un ecosistema amplio de herramientas para la validación programática de configuraciones. El backend puede implementar algoritmos de selección y verificación que comprueben automáticamente consistencia lógica, satisfacibilidad de prerequisites, exclusiones y cumplimiento de constraints. La sintaxis de Python resulta especialmente adecuada para traducir expresiones lógicas como `requires`, `excludes` o `IMPLIES`, ya sea mediante evaluaciones directas o a través de módulos que permiten construir árboles lógicos, satisfacibilidad booleana o incluso resolución por backtracking.

Otro aspecto clave es que Python permite separar con claridad el modelo (estructura estática) de las configuraciones (instancias dinámicas). A través de funciones o clases especializadas, el sistema puede generar configuraciones válidas a partir de elecciones de usuario, aplicar reglas de minimización/maximización de cardinalidades, determinar incompatibilidades, detectar violaciones y ofrecer soluciones alternativas, todo ello sin necesidad de traducir el modelo a lenguajes externos. Adicionalmente, la flexibilidad del lenguaje, junto con el respaldo de su amplio ecosistema de librerías, facilita la construcción de algoritmos que no solo derivan configuraciones de forma eficiente, sino que también las verifican mediante evaluaciones lógicas, análisis de dependencias y resolución de conflictos. Este mismo soporte permite implementar mecanismos de versionado y snapshots del modelo, conservando estados históricos y garantizando que la evolución del FM no invalide configuraciones previas, preservando así la consistencia y trazabilidad del sistema en el tiempo.

Asimismo, Python ofrece un ecosistema robusto para el desarrollo de aplicaciones web a través de frameworks como FastAPI, que permiten construir servicios Transferencia de Estado Representacional (*Representational State Transfer*) (REST)ful eficientes y escalables, lo cual

facilita exponer un motor de validación y generación de configuraciones como servicio [Ramírez, 2019]. A esto se suma la compatibilidad con herramientas modernas de persistencia de datos como SQLAlchemy y pydantic, lo que facilita la representación estructurada de información curricular y su validación automática. Esto habilita una arquitectura limpia en la que las representaciones de FM, los algoritmos de derivación y las reglas del dominio educativo quedan encapsulados y accesibles desde otros componentes de la plataforma (cliente, bases de datos, módulos administrativos, etc.).

En conjunto, estas características hacen de Python un lenguaje altamente apropiado tanto para la representación formal de modelos de características como para la generación y validación automatizada de configuraciones dentro del sistema propuesto [Ray et al., 2014]. Por su equilibrio entre simplicidad expresiva y potencia funcional, Python constituye una base tecnológica idónea para el *backend* del sistema, permitiendo acelerar el desarrollo sin sacrificar calidad, extensibilidad ni alineación con los requisitos funcionales de la solución.

1.4.2 Framework de Desarrollo

Los frameworks de desarrollo son estructuras software que proporcionan un conjunto de herramientas, patrones, módulos y convenciones que simplifican la construcción de aplicaciones [Sommerville, 2016]. Permiten acelerar la implementación de funcionalidades complejas, garantizando buenas prácticas de diseño y facilitando la mantenibilidad del software [Pressman, 2014]. Un framework adecuado reduce la repetición de código, mejora la seguridad, permite la modularización y posibilita el desarrollo escalable. En este proyecto, el framework seleccionado debe soportar concurrencia, validación, comunicación con bases de datos y un desarrollo claro basado en APIs [Gamma et al., 1995].

FastAPI será el framework *backend* adoptado para la implementación

de la plataforma, debido a su equilibrio entre simplicidad de desarrollo, alto rendimiento y robustez arquitectónica [Ramírez, 2019, Eugene and Anderson, 2023]. Su diseño se basa en los principios de asincronía nativa y validación automática de datos, permitiendo construir APIs eficientes, escalables y seguras, características fundamentales para un sistema que debe procesar configuraciones de modelos curriculares, validar restricciones y ofrecer respuestas consistentes en tiempo real.

Un factor determinante en la selección de FastAPI es su rendimiento comparable a implementaciones escritas en Go y Node.js, gracias a su construcción sobre Starlette (para la capa asíncrona) y Pydantic (para validación y serialización) [Pesce, 2021]. Esta combinación permite manejar múltiples peticiones concurrentes, realizar verificaciones de reglas e interacciones dinámicas con el motor de configuraciones sin degradación perceptible del servicio, incluso bajo cargas elevadas.

La validación de datos es un aspecto crítico del sistema propuesto, ya que el backend debe recibir elecciones de características, evaluarlas frente a cardinalidades y restricciones lógicas, y retornar configuraciones válidas o alternativas corregidas. En este sentido, FastAPI destaca por integrar modelos de datos declarativos mediante Pydantic, lo que garantiza que cada solicitud sea verificada estructural y semánticamente antes de su procesamiento [Ramírez, 2019]. Esto no solo previene errores de ejecución, sino que contribuye a la robustez del motor de configuraciones, evitando inconsistencias desde la capa de entrada.

Un argumento especialmente relevante para seleccionar FastAPI es la naturaleza intensiva en validaciones y operaciones Entrada/Salida (*Input/Output*) (I/O) que caracteriza al sistema propuesto. La plataforma no solo debe recibir configuraciones y verificar reglas, sino también consultar persistencia, aplicar evaluaciones lógicas, generar respuestas alternativas y registrar versiones de modelo, todo ello de manera concurrente y potencialmente bajo múltiples usuarios. Debido a su arquitectura asíncrona nativa basada en `async/await`, FastAPI permite manejar

este patrón de carga sin bloqueo, maximizando la eficiencia en tareas I/O y reduciendo latencias [Eugene and Anderson, 2023, Pesce, 2021]. Esto se traduce en una capacidad real de escalar, sin incurrir en cuellos de botella característicos de frameworks síncronos, lo que resulta esencial en un sistema donde las consultas y validaciones son frecuentes, complejas y potencialmente concurrentes.

Adicionalmente, FastAPI presenta una curva de aprendizaje notablemente baja, permitiendo extremo enfoque en la lógica de negocio sin sacrificar mantenibilidad. Su documentación generada automáticamente (OpenAPI/Swagger) mejora la trazabilidad del sistema y facilita la colaboración entre desarrolladores y expertos curriculares, quienes pueden visualizar endpoints, contratos de datos y respuestas sin requerir interacción directa con el código [Ramírez, 2019].

Desde el punto de vista arquitectónico, FastAPI favorece un desarrollo altamente modular, encajando de manera natural con principios de separación de responsabilidades y diseño por componentes [Richards, 2015]. Esta modularidad resulta especialmente útil para organizar servicios de modelado de FM, motor de restricciones, evaluadores de configuraciones, repositorios, controladores y almacenes de versión.

Finalmente, la compatibilidad de FastAPI con controladores de bases de datos Lenguaje de Consulta Estructurado (*Structured Query Language*) (SQL), junto con herramientas de orquestación (Docker) y Integración Continua y Despliegue Continuo (CI/CD), permite que la solución crezca de forma controlada desde prototipos académicos hasta implementaciones institucionales. En conjunto, su velocidad, asincronía, simplicidad expresiva, capacidad de validación y flexibilidad lo convierten en la opción más adecuada para el sistema curricular propuesto [Eugene and Anderson, 2023].

1.4.3 Base de Datos y Gestor de Base de Datos

Las bases de datos son sistemas diseñados para almacenar, orga-

nizar y proteger información de manera eficiente y estructurada, garantizando su disponibilidad a lo largo del tiempo. Para interactuar con ellas, se emplean gestores de bases de datos (DBMS), que proporcionan mecanismos de control, recuperación, consultas, integridad y seguridad. La selección de una base de datos adecuada depende del tipo de datos a manejar, los niveles de consistencia requeridos y los volúmenes previstos. En una plataforma como la propuesta, donde se gestionan configuraciones, versiones, restricciones y metadatos del dominio curricular, el gestor debe asegurar durabilidad, consistencia y rendimiento óptimo.

La persistencia de los modelos curriculares, configuraciones generadas, versiones históricas y metadatos asociados requiere de un sistema robusto, seguro y con garantías de consistencia transaccional. En este proyecto se selecciona PostgreSQL como gestor de base de datos relacional, debido a su madurez, fiabilidad y su capacidad de manejar estructuras complejas con fuertes requisitos de integridad referencial [PostgreSQL Global Development Group, 2024, Momjian, 2001]. PostgreSQL destaca por su cumplimiento estricto del estándar SQL, su modelo Atomicidad, Consistencia, Aislamiento, Durabilidad (*Atomicity, Consistency, Isolation, Durability*) (ACID), y su eficiente gestión de transacciones, elementos imprescindibles para representar relaciones jerárquicas entre características, restricciones cross-tree, y snapshots versionados del modelo sin comprometer la coherencia global del sistema [Momjian, 2001].

Además, PostgreSQL ofrece capacidades avanzadas como índices compuestos, consultas optimizadas, vistas materializadas, triggers y procedimientos almacenados, que resultan útiles para procesar datos curriculares con alto grado de estructuración [PostgreSQL Global Development Group, 2024]. El motor también soporta tipos de datos Notación de Objetos JavaScript (*JavaScript Object Notation*) (JSON) y JSON Binario (*JSON Binary*) (JSONB), permitiendo almacenar representaciones

semiestructuradas de configuraciones y estados transitorios, por lo que proporciona flexibilidad para almacenar tanto esquemas estáticos como snapshots dinámicos relacionados con la evolución del Modelo de Características [PostgreSQL Global Development Group, 2024].

Adicionalmente, PostgreSQL se integra de forma natural con SQLAlchemy como Mapeo Objeto-Relacional (*Object-Relational Mapping*) (ORM) dentro del backend desarrollado en FastAPI, facilitando el diseño declarativo de entidades, la gestión del ciclo de persistencia, la validación de relaciones y la migración segura del esquema. Esta integración favorece una arquitectura limpia y escalable, desacoplando las capas de datos, servicios y lógica de negocio.

Para la administración visual del motor se empleará pgAdmin, interfaz ampliamente utilizada para la gestión de bases de datos PostgreSQL. Mediante esta herramienta es posible inspeccionar tablas, ejecutar consultas SQL, visualizar índices y constraints, supervisar cargas de trabajo y realizar respaldos de forma controlada. En el contexto de la solución propuesta, pgAdmin permite auditar el estado de los modelos almacenados, verificar reglas impuestas, revisar configuraciones generadas y garantizar la trazabilidad del sistema.

En conjunto, la elección de PostgreSQL como sistema de persistencia y pgAdmin como herramienta administrativa proporciona una base confiable y alineada con las necesidades de integridad, consistencia y trazabilidad del sistema curricular modelado [Momjian, 2001, PostgreSQL Global Development Group, 2024].

1.4.4 Contenerización y Orquestación

La contenerización es una tecnología que permite empaquetar aplicaciones junto con todas sus dependencias en unidades aisladas, reproducibles y portables llamadas contenedores. Esto garantiza que una aplicación se ejecute de la misma manera en cualquier entorno, eliminando problemas de incompatibilidad. Para ello se emplean herramien-

tas especializadas, donde Docker se ha consolidado como estándar industrial. Por su parte, la orquestación permite administrar múltiples contenedores, asegurando coordinación, escalabilidad y resiliencia; funciones que normalmente se alcanzan mediante tecnologías como Docker Compose o Kubernetes. Este enfoque es crucial para despliegues confiables, mantenibles y escalables.

Para garantizar la portabilidad, reproducibilidad y facilidad de despliegue del sistema propuesto, se adoptará Docker como tecnología base de contenedorización [Boettiger, 2015, Inc., 2024a]. Docker permite encapsular los servicios de la aplicación —incluyendo el backend FastAPI, la base de datos PostgreSQL, la herramienta administrativa pgAdmin y otros servicios de utilidad o futuros módulos asociados— dentro de entornos aislados y autocontenidos. Esto elimina problemas de dependencias, divergencias de entorno y configuraciones inconsistentes entre despliegues locales y productivos [Boettiger, 2015]. La capacidad de reproducir una imagen de forma determinística contribuye directamente a la robustez y mantenibilidad de la solución.

La contenedorización resulta especialmente útil para aplicaciones multicapa como la propuesta, donde distintos servicios deben colaborar manteniendo canales de comunicación estables. Docker permite garantizar uniformidad tanto en las dependencias del backend, como versiones de librerías, configuración del servidor de aplicación y parámetros del engine de base de datos, disminuyendo errores derivados de diferencias de entorno y facilitando diagnósticos [Inc., 2024a].

Sobre Docker se construye Docker Compose, herramienta de orquestación ligera que permite describir, mediante un único archivo declarativo, toda la topología de servicios del sistema [Inc., 2024a]. Esto incluye la configuración de redes internas, gestión de persistencia en volúmenes, variables de entorno, reglas de dependencias entre contenedores, puntos de exposición de puertos, así como políticas de reinicio. En el contexto del sistema de modelado curricular, Docker Compose facilita levantar

el entorno completo de desarrollo o producción con un solo comando, garantizando escalabilidad y reduciendo tiempos de despliegue o recuperación ante fallos [Boettiger, 2015].

Un beneficio adicional de esta estrategia es su contribución a las prácticas de CI/CD. Gracias a la naturaleza declarativa y reproducible de los contenedores, el sistema puede desplegarse automáticamente mediante pipelines controlados sin intervención manual, asegurando que las versiones del modelo curricular, las configuraciones generadas y los snapshots almacenados sean accesibles en entornos iguales o equivalentes entre sí [Inc., 2024a].

De esta forma, el uso conjunto de Docker y Docker Compose aporta portabilidad, eficiencia en despliegue, coherencia entre entornos y una base técnica sólida para escalar la solución futura en producción.

1.4.5 Almacenamiento en Memoria, Caching y Mensajería

El almacenamiento en memoria es una técnica que permite guardar temporalmente datos en memoria RAM para acelerar el acceso a información que se consulta de forma recurrente, evitando operaciones de disco o consultas de base de datos innecesarias. El caching se basa precisamente en optimizar velocidad, reduciendo latencias y mejorando el rendimiento global del sistema. Existen herramientas especializadas que permiten gestionar este almacenamiento temporal, soportar estructuras clave-valor y facilitar comunicación entre servicios. Redis destaca entre ellas por su eficiencia, simplicidad y capacidades adicionales como mensajería, pub/sub y persistencia configurable, lo que la convierte en una pieza clave para arquitecturas modernas.

Redis será incorporado como componente clave para el almacenamiento temporal y procesamiento eficiente de información crítica durante las fases de generación y validación de configuraciones curriculares [Carlson, 2013]. Su naturaleza de base de datos en memoria permite alcanzar tiempos de respuesta extremadamente bajos, lo cual

resulta particularmente valioso en un sistema donde las operaciones de verificación de restricciones, derivación de configuraciones válidas y resolución de incompatibilidades pueden ejecutarse de forma intensiva y repetitiva.

A diferencia de un motor relacional tradicional, Redis está optimizado para operaciones clave–valor y estructuras avanzadas como listas, conjuntos, mapas hash y sorted sets [Carlson, 2013]. Estas estructuras permiten modelar transitoriamente elementos como:

- configuraciones candidatas generadas por el usuario,
- listas de incompatibilidades detectadas,
- descriptores internos del FM,
- estados de sesión durante la exploración de alternativas,
- cachés de resultados de validaciones repetitivas.

Este almacenamiento en memoria reduce significativamente la latencia, lo que contribuye a experiencias interactivas y a la capacidad del sistema de responder en tiempo real a decisiones del usuario (por ejemplo, activar o desactivar características, consultar prerequisites, etc.) [Carlson, 2013].

Otro aspecto clave es que Redis permite persistencia opcional mediante snapshots (Base de Datos Relacional (*Relational Database*) (RDB)), lo que lo convierte en un aliado directo del soporte de versionado de modelos y configuraciones [Carlson, 2013]. Es decir, puede almacenar estados intermedios del proceso de modelado curricular sin interferir con la base de datos principal, permitiendo restaurar fácilmente versiones anteriores o explorar ramas alternativas del mismo modelo.

Además, Redis implementa un sistema de Time-To-Live (Tiempo de Vida (*Time To Live*) (TTL)), lo que permite gestionar la caducidad automática de configuraciones temporales, sesiones y cálculos ya procesados. Esta funcionalidad contribuye a la integridad global del sistema y evita acumulación de datos residual [Carlson, 2013].

Finalmente, una característica determinante para su elección es su excelente integración con FastAPI, permitiendo aprovechar plenamente la naturaleza asíncrona del backend [Ramírez, 2019]. Gracias a librerías como aioredis, las operaciones de lectura/escritura se realizan sin bloquear el event loop, lo que maximiza la concurrencia cuando múltiples usuarios exploran configuraciones simultáneamente.

En síntesis, Redis aporta velocidad, soporte a cálculos temporales complejos, persistencia selectiva, alta concurrencia y facilidad de integración con el framework propuesto, consolidándose como la pieza ideal para manejar la lógica dinámica del sistema.

1.4.6 Almacenamiento de Archivos y Objetos

El almacenamiento persistente de archivos, versiones históricas de modelos y resultados derivados de las evaluaciones requiere una tecnología que ofrezca acceso distribuido, tolerancia a fallos, escalabilidad y compatibilidad con estándares industriales. Para satisfacer esta necesidad se incorporará **MinIO**, un sistema de almacenamiento de objetos compatible con la Interfaz de Programación de Aplicaciones (*Application Programming Interface*) (API) de Amazon Simple Storage Service (Amazon S3) (S3), ampliamente adoptado en entornos de producción y servicios en la nube [Min, 2024].

MinIO ofrece una interfaz uniforme para almacenar objetos binarios, documentos, snapshots de modelos y resultados asociados al proceso de validación. Su compatibilidad con S3 permite integrar la plataforma con herramientas estándar sin requerir dependencias propietarias [Adler and Zhang, 2023]. Esta característica asegura interoperabilidad con sistemas externos, posibilitando que módulos de análisis, auditoría o visualización operen sobre un backend de almacenamiento robusto y escalable.

En el contexto de la solución propuesta, MinIO será empleado para almacenar:

- snapshots persistentes de versiones del FM,
- configuraciones válidas generadas por los motores SAT/Satisfacibilidad Módulo Teorías (*Satisfiability Modulo Theories*) (SMT),
- artefactos derivados del análisis estructural,
- archivos de auditoría e informes,
- registros de pruebas y diagnósticos.

A diferencia del almacenamiento en memoria provisto por Redis, MinIO garantiza persistencia a largo plazo, redundancia y recuperación confiable de información. Su arquitectura distribuida permite configurar réplicas, dispersión de datos y mecanismos de resiliencia sin sacrificar rendimiento [Adler and Zhang, 2023]. Además, sus políticas de control de acceso simplifican la gestión de permisos, evitando inconsistencias o pérdida de trazabilidad a lo largo del ciclo de vida del currículo.

Desde el punto de vista operativo, MinIO permite integrarse con Docker y Docker Compose de manera directa, facilitando el despliegue, escalado y administración sin requerir configuración adicional. Esto habilita una solución completa de almacenamiento distribuido que puede operar tanto en entornos locales (on-premise) como en infraestructuras institucionales o nube privada.

En síntesis, MinIO aporta durabilidad, eficiencia, compatibilidad, escalabilidad y resiliencia, consolidándose como la opción adecuada para soportar el almacenamiento persistente de artefactos curriculares y resultados analíticos del sistema propuesto.

1.4.7 Procesamiento Asíncrono

El procesamiento asíncrono permite ejecutar tareas en segundo plano sin bloquear la operación principal del sistema, lo cual es esencial cuando existen tareas costosas, como validaciones complejas, consultas externas, cálculos intensivos o envío de notificaciones. En el

ecosistema de Python existen herramientas especializadas para administrar colas de trabajo, distribuir tareas y realizar ejecución diferida. Entre ellas, Celery se ha posicionado como la solución estándar para orquestar tareas asíncronas, ofreciendo escalabilidad, distribución y tolerancia a fallos. Su integración con brokers como Redis permite construir sistemas altamente eficientes, modulares y no bloqueantes.

En la arquitectura propuesta, Celery desempeñará un rol esencial como orquestador de tareas asíncronas, permitiendo que operaciones complejas, lentas o intensivas en cómputo se ejecuten fuera del ciclo de solicitud-respuesta del API, mejorando sustancialmente la experiencia del usuario y la disponibilidad del sistema [Project, 2024].

Dado que el proceso de generación y validación de configuraciones curriculares incluye fases donde se deben:

- Recorrer árboles completos del Feature Model,
- Validar múltiples restricciones cruzadas,
- Propagar efectos lógicos por dependencias,
- Explorar configuraciones alternativas válidas,
- Verificar versiones y retrocompatibilidad,

Resulta natural que estas tareas no se ejecuten directamente en una petición HTTP, ya que podrían bloquear el servidor o provocar tiempos de respuesta superiores a lo aceptable. Celery permite delegar estos procesos a “workers” paralelos que pueden ejecutarse en segundo plano, liberando a FastAPI para seguir atendiendo solicitudes sin congestión.

La integración de Celery con Redis (como message broker) permite que las tareas se publiquen, distribuyan, encolen, ejecuten y documenten de forma confiable. Asimismo, Redis funge como backend de resultados, permitiendo al sistema:

- Consultar el estado de una ejecución,

- Recuperar resultados parciales o finales,
- Cancelar o reprogramar tareas,
- Mostrar progresos al usuario.

Otra ventaja crítica es que Celery soporta escalamiento horizontal, lo cual es relevante para tu sistema, en el cual múltiples usuarios pueden realizar evaluaciones simultáneamente sobre modelos distintos o versiones diferentes. Con más carga, simplemente se añaden más workers, sin necesidad de cambios estructurales [Project, 2024].

Desde el punto de vista de robustez, Celery ofrece herramientas para:

- Reintentos automáticos ante fallos,
- Límite de concurrencia,
- Priorización de colas,
- Periodicidad (con Celery Beat) para ejecutar tareas programadas.

Esto permite automatizar procesos como:

- Generación periódica de snapshots de Feature Models,
- Validaciones nocturnas de consistencia,
- Limpieza de configuraciones expiradas,
- Backups de estados intermedios.

Finalmente, Celery es altamente compatible con FastAPI y Python, manteniendo coherencia tecnológica con la decisión de usar un stack asíncrono y un modelo de validación compleja [Project, 2024, Ramírez, 2019]. Su adopción garantiza que la capa lógica pesada no degrade la utilidad interactiva del sistema, sino que se ejecute de forma paralela, programada y resistente. La siguiente tabla 1.5 respalda la justificación de por qué adicionar celery al sistema.

Cuadro 1.5: Tabla Comparativa: FastAPI vs FastAPI + Celery. Fuente: Elaboración propia.

Criterio	Solo FastAPI	FastAPI + Celery
Manejo de operaciones pesadas	Bloquea el servidor si la tarea tarda mucho	Delegación a workers externos sin bloqueo
Experiencia del usuario	Tiempos de respuesta largos para validaciones	Respuestas inmediatas + seguimiento del progreso
Escalabilidad	Limitada al número de workers del servidor	Escalamiento horizontal agregando workers Celery
Naturaleza de carga	Orientada a I/O y sincronización	Orientada a cómputo intensivo y tareas diferidas
Validación de modelos complejos	Puede agotar recursos del servidor	Optimización mediante workers paralelos
Generación de snapshots o versiones	Bajo rendimiento si se ejecuta en el request	Tareas programadas y automáticas (Celery Beat)
Robustez ante fallos	Fallos interrumpen solicitudes activas	Reintentos automáticos e independientes
Concurrencia	Limitada	Altamente concurrente
Distribución de carga	Manual, difícil de gestionar	Balance natural mediante colas y brokers
Estado de procesos largos	No persistente, no rastreable	Persistente, rastreable, recuperable
Integración con Redis	Opcional	Ideal, Redis como broker y backend

Continúa en la siguiente página

Cuadro 1.5 – Continuación de la página anterior

Criterio	Solo FastAPI	FastAPI + Celery
Mantenimiento	Monolítico	Arquitectura de-sacoplada y modular

Esta comparación evidencia que mientras FastAPI permite construir servicios web altamente eficientes para operaciones sin bloqueo y validaciones ligeras, la incorporación de Celery habilita un procesamiento robusto y distribuido ideal para la naturaleza intensiva de la manipulación y validación de Feature Models, generando configuraciones, snapshots y cálculos alternativos sin afectar la disponibilidad del sistema.

1.4.8 Algoritmos, Motor de Reglas y Validación

Para garantizar la consistencia, completitud y validez de las configuraciones generadas a partir del Modelo de Características (FM), la plataforma integrará un conjunto de algoritmos especializados, combinando enfoques deterministas y heurísticos según la naturaleza del problema a resolver.

En primer lugar, se utilizará **SymPy** como motor base para la representación simbólica y evaluación lógica de restricciones asociadas al FM [Meurer, 2017]. SymPy permite manipular expresiones booleanas, aplicar simplificaciones, detectar contradicciones y verificar satisfacibilidad, lo que resulta esencial para la evaluación de relaciones *cross-tree* como *requires*, *excludes*, *implies* y restricciones de cardinalidad. Su uso asegura una verificación formal, basada en lógica proposicional y álgebra simbólica, manteniendo precisión y trazabilidad en los análisis [Meurer, 2017].

De manera complementaria, la plataforma empleará **NetworkX** para modelar y procesar las dependencias entre características como grafos dirigidos, permitiendo ejecutar algoritmos de recorrido, detección de ci-

clos, análisis de conectividad, propagación de condiciones y determinación de componentes fuertemente conexas [Hagberg et al., 2008]. Este enfoque orientado a grafos habilita la inspección estructural del FM, la detección temprana de inconsistencias y la aplicación eficiente de validaciones transversales.

En escenarios donde la validación determinista no sea suficiente para explorar el espacio de configuraciones posibles —especialmente cuando existan alternativas múltiples y restricciones complejas— se utilizarán técnicas heurísticas como **algoritmos genéticos**. Estos permiten buscar soluciones aproximadas optimizando criterios como completitud, compatibilidad y minimización de violaciones. Su empleo resulta valioso para sugerir configuraciones válidas, proponer versiones corregidas y explorar variantes del FM bajo metas específicas.

Si bien el sistema incorpora herramientas como SymPy para análisis simbólico, NetworkX para la evaluación estructural del modelo y algoritmos genéticos para la exploración heurística del espacio de soluciones, estas tecnologías no sustituyen un motor formal de satisfacibilidad. Por su naturaleza, SymPy resulta adecuado para verificar restricciones locales o detectar inconsistencias obvias, pero no está diseñado para resolver problemas de variabilidad a gran escala. De igual forma, los algoritmos evolutivos permiten sugerir configuraciones o completar decisiones, aunque sus resultados son aproximados. Por ello, la plataforma complementa este enfoque híbrido con motores lógicos especializados basados en SAT/SMT para garantizar corrección formal y escalabilidad en los análisis.

Para validar y generar configuraciones curriculares consistentes, se adoptó por un motor lógico basado en técnicas de satisfacibilidad booleana (SAT) y resolución de restricciones (CSP). Estas técnicas son exactas, determinísticas y ampliamente empleadas en entornos SPL industriales. Librerías como PySAT (python-sat) y Z3 Solver permiten representar cardinalidades, dependencias, exclusiones y restricciones

complejas entre características, garantizando un análisis consistente independientemente del tamaño del modelo.

Para la validación formal del FM se hará uso de **PySAT**, un *toolkit* industrial que integra múltiples solucionadores SAT de alto rendimiento (MiniSAT, Glucose, Lingeling, Maple, entre otros). PySAT permite formular restricciones curriculares como cláusulas booleanas y delegar su resolución a técnicas probadas a nivel de investigación y producción [Zhao et al., 2018]. Su uso mejora la escalabilidad del sistema, ya que permite manejar modelos con miles de dependencias de forma eficiente, además de habilitar análisis avanzados como detección de dependencias redundantes, pruebas de satisfacibilidad incremental y extracción de explicaciones (UNSAT cores), funcionalidad útil para asistir al usuario en la corrección de configuraciones inválidas.

Los solucionadores **SAT-based** han demostrado ser una técnica altamente efectiva en Ingeniería de Líneas de Productos (SPL) para la verificación automática de configuraciones [Benavides et al., 2010c]. Su aplicación consiste en reducir el Modelo de Características (FM) a una fórmula booleana y comprobar su satisfacibilidad mediante algoritmos especializados. Este enfoque es determinista, completo y garantiza la detección de inconsistencias globales, no sólo locales. A diferencia de técnicas heurísticas, los solucionadores SAT permiten decisiones exactas, prueba de imposibilidad, generación de configuraciones alternativas, evaluación de cardinalidades y análisis de impacto ante cambios, aportando rigor matemático y soporte computacional sólido para la variabilidad curricular.

Mientras los solucionadores SAT determinan si una configuración es válida o no, el uso de técnicas **Max-SAT** permite optimizar soluciones, seleccionar variantes preferidas y resolver conflictos minimizando el número de violaciones. El solucionador Z3, desarrollado por Microsoft Research, soporta **Max-SAT**, SMT (Satisfiability Modulo Theories) y razonamiento híbrido, lo que lo convierte en una herramienta

idónea para el análisis curricular [De Moura and Bjørner, 2008]. Esto permite no sólo verificar configuraciones, sino sugerir alternativas cercanas a las decisiones originales del usuario, resolver conflictos automáticamente y explorar escenarios de “mejor esfuerzo” cuando no existe una solución exacta. En términos prácticos, Max-SAT habilita un sistema inteligente que auxilia al gestor curricular en decisiones complejas, proporcionando soluciones justificadas y transparentes.

1.4.9 Gestor de dependencias

Para la gestión eficiente de dependencias, aislamiento de entornos y versionado reproducible de librerías, la plataforma adoptará **Poetry** y **UV** como gestores complementarios [authors, 2024].

Poetry permite mantener un control declarativo de las dependencias del proyecto mediante archivos `pyproject.toml`, garantizando la trazabilidad del stack tecnológico y la compatibilidad de versiones. Su mecanismo de resolución de dependencias asegura consistencia en los entornos de desarrollo y producción, reduciendo incidencias asociadas a incompatibilidades entre librerías [authors, 2024].

Además, Poetry facilita la generación de entornos virtuales aislados, el bloqueo de versiones exactas (`poetry.lock`), la publicación del proyecto como paquete y la automatización de scripts relacionados con el ciclo de vida del backend.

En conjunto con Poetry, se utilizará **UV** como herramienta moderna para instalación ultrarrápida de dependencias y creación de entornos [Marsh, 2024]. UV destaca por su rendimiento de orden de magnitud superior a herramientas tradicionales, lo cual mejora tiempos de despliegue, entornos CI/CD y operaciones de reconstrucción.

La combinación de Poetry y UV permite una gestión robusta, reproducible y eficiente de dependencias, alineada con las necesidades de mantenibilidad, escalabilidad y automatización del sistema [authors, 2024, Marsh, 2024].

1.4.10 Herramienta para el control de versiones

Los sistemas de control de versiones más populares son Git, Subversion y Mercurial. Se eligió Git debido a que es un sistema de control de versiones distribuido utilizado en el desarrollo de software que rastrea las modificaciones realizadas en el código fuente. Git, a diferencia de los sistemas de control de versiones centralizados, permite que varios desarrolladores colaboren de forma independiente en un solo proyecto al permitirles mantener repositorios locales separados que se sincronizan con un repositorio remoto. Los repositorios, que contienen los archivos del proyecto y el historial de cambios, son esenciales para el funcionamiento de Git [Ponuthorai and Loeliger, 2022]. Una confirmación en un repositorio es una instantánea del proyecto en ese momento específico; cada confirmación se distingue por un *hash* distinto y va acompañada de un mensaje de explicación. Las ramas permiten a los desarrolladores trabajar de forma independiente en varias funciones o correcciones [Loeliger and McCullough, 2012].

Mientras que otras ramas se utilizan para el desarrollo, el código estable generalmente se encuentra en la rama principal, también conocida como *main*. La reorganización y la combinación son dos formas en que los cambios de las ramas se pueden combinar en la rama principal, cada una con una función distinta para preservar el historial del proyecto. Se crea una copia local de un repositorio clonándolo, y los cambios locales se sincronizan con un repositorio remoto mediante la inserción y la extracción [Ghodke and Chavan, 2024]. El área de preparación sirve como zona de influencia donde se realizan modificaciones antes de la confirmación. Los submódulos, *cherry-pick* y Git *hooks* son ejemplos de funciones avanzadas que proporcionan más personalización y control. En resumen, Git permite llevar un historial exhaustivo de cambios, ramificación para desarrollo de nuevas características, fusiones controladas, auditoría de modificaciones y recuperación de estados previos del sistema. Los conocidos servicios de alojamiento de Git, como GitHub,

GitLab y Bitbucket, ofrecen plataformas para la revisión de código, la gestión de repositorios y la integración de herramientas, que promueven el desarrollo colaborativo. El uso de Git mejora los procesos de desarrollo al facilitar el trabajo en equipo y la gestión de código efectivos [Ponuthorai and Loeliger, 2022].

GitHub es una plataforma de alojamiento de código basada en la web que utiliza Git como tecnología subyacente para el control de versiones. Actúa como un servicio de repositorio remoto centralizado, ampliando las capacidades de Git con herramientas de colaboración, gestión de proyectos y características sociales. Como una de las plataformas más populares para el desarrollo de software colaborativo, GitHub permite a los equipos alojar repositorios públicos y privados, facilitando la revisión de código mediante *pull requests*, el seguimiento de problemas con *issues* y la automatización de flujos de trabajo mediante GitHub Actions. Su interfaz gráfica y sus herramientas de gestión hacen que las operaciones de Git, como la fusión de ramas y la resolución de conflictos, sean más accesibles para los desarrolladores, al tiempo que proporcionan integraciones robustas con herramientas de CI/CD y servicios de terceros. GitHub también fomenta la comunidad *open-source* mediante funciones como la bifurcación (*forking*) de repositorios y las contribuciones externas, consolidándose como un entorno integral para el ciclo de vida del desarrollo de software [Inc., 2024b, Ghodke and Chavan, 2024].

1.4.11 Lenguaje de modelados

El lenguaje de modelado se refiere a un lenguaje formal utilizado para crear modelos que representan sistemas, procesos o estructuras de datos. Estos lenguajes proporcionan un conjunto de reglas sintácticas y semánticas que facilitan la comprensión de fenómenos complejos de manera estructurada [Rumbaugh et al., 2004].

El lenguaje de modelado a utilizar es el Lenguaje de Modelado

Unificado (Lenguaje de Modelado Unificado (*Unified Modeling Language*) (UML)), este lenguaje ofrece un estándar para describir un plano del sistema, incluyendo aspectos conceptuales tales como procesos de negocios, funciones del sistema, expresiones de lenguajes de programación, esquemas de bases de datos y componentes reutilizables. Este lenguaje de modelo es el seleccionado para especificar o describir métodos o procesos, definir el sistema y sus artefactos para la documentación y construcción de la aplicación web a desarrollar [Obj, 2017].

1.4.12 Herramienta CASE

Una herramienta de la ingeniería de software asistida por computadoras (CASE) es un programa informático destinado a aumentar la productividad durante todo el proceso de desarrollo de software reduciendo el coste de estas en términos de tiempo y de dinero [Pressman, 2014]. Según Sommerville, estas herramientas nos pueden ayudar en todos los aspectos del ciclo de vida de desarrollo del *software* en tareas como el diseño de proyectos, cálculo de costes, implementación de parte del código automáticamente con el diseño dado, compilación automática, documentación o detección de errores, automatizar actividades de análisis, mejorando así la productividad y calidad del *software*. Durante el diseño de la solución se empleó la herramienta CASE: Visual Paradigm en su versión 16.2 [Sommerville, 2016].

La herramienta Visual Paradigm es reconocida mundialmente por sus soluciones de software para la transformación de negocio. Propicia un conjunto de ayudas para el desarrollo de programas informáticos, desde la planificación, pasando por el análisis y el diseño, hasta la generación de código. Soporta los principales estándares de la industria tales como el Lenguaje de Modelado Unificado (UML), la notación de Modelado de Procesos de Negocio (Notación de Modelado de Procesos de Negocio (*Business Process Model and Notation*) (BPMN)) y un sinfín de diagramas. Por lo que Visual Paradigm es una herramienta Ingeniería

de Software Asistida por Computadora (*Computer-Aided Software Engineering*) (CASE) integral y flexible para el modelo y gestión en el desarrollo de software, procesos de negocio y arquitectura empresarial [Ltd., 2024].

1.4.13 Herramientas para diseñar y ejecutar pruebas

Para garantizar la calidad y correcto funcionamiento del sistema propuesto, se emplearán herramientas especializadas tanto para pruebas de API como para pruebas unitarias del código.

Postman será utilizado como plataforma de desarrollo y pruebas de APIs [Inc., 2024c]. Esta herramienta permite diseñar, documentar y probar endpoints RESTful de forma interactiva, facilitando la verificación de contratos de datos, validación de respuestas, gestión de variables de entorno y automatización de colecciones de pruebas. Postman resulta esencial para validar el comportamiento del backend FastAPI, asegurando que las operaciones de generación y validación de configuraciones curriculares respondan correctamente bajo distintos escenarios [Inc., 2024c].

Complementariamente, se utilizará **pytest** como framework para la ejecución de pruebas unitarias en Python [Krekel and contributors, 2024]. Pytest destaca por su sintaxis clara, capacidad de parametrización, gestión de fixtures y generación de reportes detallados, lo que facilita la escritura y mantenimiento de pruebas automatizadas. Su integración con el ecosistema Python permite validar lógica de negocio, algoritmos de validación, operaciones sobre modelos de características y comportamiento de componentes aislados del sistema [Krekel and contributors, 2024].

La combinación de Postman para pruebas de integración a nivel de API y pytest para pruebas unitarias proporciona una cobertura completa del sistema, garantizando robustez tanto en la interfaz externa como en la lógica interna.

1.4.14 Entorno Integrado de Desarrollo

Los Entornos Integrados de Desarrollo (Entorno de Desarrollo Integrado (*Integrated Development Environment*) (IDE)) son las herramientas que todo desarrollador debe tener a mano. Permiten editar y gestionar código fuente en diversos lenguajes de programación y ofrecen múltiples herramientas para facilitar el trabajo y aumentar la productividad, pues permiten depurar, resaltar sintaxis, autocompletado, integración con control de versiones y la refactorización. Por lo general cada IDE está asociado a un único lenguaje de programación o a un conjunto pequeño de lenguajes, cuyo factor provoca que la gran mayoría de desarrolladores opten por otras alternativas, ya que en proyectos donde el uso de múltiples lenguajes es evidente, puede ser un obstáculo [IBM Corporation, 2023].

Se escogió como entorno de desarrollo Visual Studio Code [Corporation, 2025a], cuya herramienta es no convencional pero muy usada por muchos programadores, siendo señalado como el editor de código más popular entre proyectos de código abierto (*open source*) [Ramel, 2021]. El creador del editor de código multiplataforma, VSCode, es Microsoft. Este software es gratuito y de código abierto. Incluye soporte para la depuración, control integrado de Git, resaltado de sintaxis, finalización inteligente de código, fragmentos y refactorización de código. Además, su nivel de personalización permite adaptar el entorno a las preferencias de cada programador mediante temas, atajos y configuraciones avanzadas [Corporation, 2025b].

VSCode presenta extensiones importantísimas que permite adaptar el entorno de desarrollo para una enorme infinidad de tareas, tecnologías y lenguajes. Este conjunto, prácticamente ilimitado, es totalmente accesible dentro del propio editor. Tiene soporte integrado para aplicaciones web; por lo tanto, se puede llevar a cabo en este entorno de trabajo la codificación de la aplicación a desarrollar con las tecnologías Fast API y Python. VSCode es considerado por brindar eficiencia y agili-

dad en la programación, pues funciona muy bien en incluso equipos con recursos limitados; además, los desarrolladores lo aprecian porque su interfaz de usuario es muy intuitiva y permite comenzar, prácticamente, a trabajar sin necesidad de alguna experiencia [Corporation, 2025a,b].

1.5. Propuesta de Solución

La solución propuesta consiste en el diseño e implementación de un servicio *backend* especializado para la gestión de FM aplicados al dominio curricular. Esta solución permitirá representar formalmente planes de estudio, definir variabilidad entre módulos, establecer dependencias, restricciones y rutas formativas alternativas, así como derivar configuraciones válidas que representen itinerarios personalizados. En esencia, el sistema habilita una ingeniería curricular basada en la lógica SPL, donde los planes de estudio se conciben como familias de productos y cada configuración curricular corresponde a un producto derivado.

El servicio abordará todas las fases necesarias para este proceso: diseño del modelo curricular (equivalente al análisis de dominio), definición de restricciones y cardinalidades (gestión de variabilidad), validación automatizada de configuraciones (ensamblaje de productos) y versionamiento del modelo (gestión de activos). Para ello se implementará un motor central que permita representar jerarquías de características, aplicar reglas transversales, procesar requerimientos, exclusiones e implicaciones, y generar configuraciones consistentes, detectando violaciones y proponiendo alternativas.

El sistema incorporará funcionalidades que responden directamente a los retos del dominio educativo: evitar inconsistencias curriculares, detectar prerrequisitos no satisfechos, prevenir combinaciones inviables, explorar rutas posibles, permitir adaptaciones por niveles, perfiles o competencias y almacenar snapshots versionados que faciliten la evolución

del plan de estudios sin invalidar configuraciones previas. Este enfoque no se limita a documentar un currículo, sino que lo convierte en un modelo computable y verificable, habilitando su reutilización y variación controlada.

Finalmente, este epígrafe describe la arquitectura conceptual, los procesos funcionales, los componentes principales del sistema, los mecanismos de validación y generación de configuraciones, así como su flujo integral desde la definición del modelo hasta la derivación de itinerarios formativos personalizados. De este modo, se establece una propuesta técnica y metodológica sólida, capaz de aportar rigor, flexibilidad y trazabilidad a la gestión curricular.

1.5.1 Modelo SPL y su Correspondencia con la Propuesta

La propuesta de solución se alinea conceptualmente con el enfoque de Líneas de Productos de Software (SPL), particularmente con el modelo de doble ciclo de vida (Two-Lifecycle Model), donde se distinguen claramente dos etapas: Ingeniería de Dominio (desarrollo de activos reutilizables) e Ingeniería de Aplicación (derivación de productos específicos a partir de dichos activos). En el contexto curricular, este enfoque permite abstraer el modelo del plan de estudios como un activo general, del cual se pueden derivar múltiples configuraciones personalizadas que responden a perfiles, competencias o rutas formativas diferentes.

En la fase de Ingeniería de Dominio, el sistema propuesto representa formalmente el Modelo de Características curriculares (FM) mediante estructuras jerárquicas, relaciones cross-tree, cardinalidades y restricciones lógicas. Esta fase captura la comunalidad del dominio educativo (asignaturas, prerrequisitos, módulos obligatorios) y su variabilidad (alternativas, secuencias opcionales, rutas especializadas). Dicho FM constituye la base reutilizable que puede evolucionar mediante versionamiento, manteniendo trazabilidad y preservando consistencia histórica. En términos SPL, este modelo representa el conjunto de “core assets”

curriculares.

En la fase de Ingeniería de Aplicación, el FM se utiliza para derivar configuraciones curriculares válidas, equivalentes a productos dentro de una familia. El sistema interpreta selecciones de características realizadas por el usuario, verifica restricciones, detecta inconsistencias y propone soluciones alternativas. De esta forma, la implementación curricular final (como un itinerario formativo, malla o perfil de especialización) se construye mediante ensamblaje controlado de variabilidad, garantizando que cualquier combinación derivada cumpla las reglas definidas en el modelo base.

La propuesta incorpora mecanismos que facilitan esta gestión dual: validación automática, versionamiento del modelo, almacenamiento de snapshots, trazabilidad y gestión de restricciones. En consecuencia, la plataforma no actúa únicamente como un repositorio de información académica, sino como una infraestructura que operacionaliza principios SPL en un contexto educativo, permitiendo generar itinerarios formativos parametrizados, reproducibles y verificables. Esta integración del paradigma SPL habilita un proceso sistemático de personalización curricular, manteniendo coherencia, calidad y reutilización estructurada.

1.5.1.1 Justificación del uso de los FM

La verdadera potencia del FM radica en que, una vez formalizado, se convierte en un modelo lógico-matemático que define explícitamente el espacio de diseño o el conjunto de configuraciones válidas (2^n) que se pueden generar (donde n es el número de características). Esta formalidad permite realizar análisis automatizados esenciales mediante herramientas conocidas como solvers de satisfacibilidad (SAT Solvers) o restricciones, permitiendo la Validación de Consistencia, que se encarga de Determinar si un FM es consistente, es decir, si existe al menos una configuración válida, la Identificación de Características Muertas (Dead Features), la cual permite encontrar las Características que nunca pueden ser seleccionadas debido a la severidad de las restricciones

y posibilitan una Configuración Guiada, que asiste al usuario en la selección de opciones para garantizar que el producto final (el Plan de Estudios) sea siempre válido.

Aunque existen otros modelos de variabilidad (como las tablas de decisión o los diagramas UML de variabilidad), el FM es el formalismo preferido por su simplicidad visual y su potencia analítica. A diferencia de modelos como las tablas, que pueden volverse inmanejables al aumentar el número de características, el FM provee una visión de alto nivel y, simultáneamente, una base lógica binaria robusta para el análisis automatizado de la consistencia (el propósito crucial en la propuesta de solución).

En el contexto de la plataforma web propuesta, el Modelo de Características es el artefacto central. El Módulo Frontend tendrá el propósito de funcionar como un Editor Gráfico de FMs, traduciendo la notación visual (árboles y símbolos) a la estructura lógica subyacente. Esto permitirá a los gestores académicos:

1. **Modelar:** Crear y modificar visualmente la estructura curricular (árbol de asignaturas).
2. **Validar:** Enviar el modelo lógico para su análisis, asegurando que no existan inconsistencias (rutas académicas imposibles) en el currículo.
3. **Configurar:** Utilizar el modelo como base para generar planes de estudio válidos, adaptados a perfiles específicos.

1.5.1.2 La Analogía Formal Curricular

La aplicación de los Modelos de Características (FM) a la gestión y diseño curricular constituye el principal aporte metodológico de esta investigación. Esta extensión se fundamenta en la premisa de que la variabilidad, concepto central de la Ingeniería de Líneas de Productos

de Software (SPL), es intrínseca y necesaria en los Planes de Estudio modernos de la enseñanza superior.

Para fundamentar la propuesta, es imprescindible establecer una analogía formal y coherente que permita sustituir los constructos de SPL por sus equivalentes en el dominio educativo, elevando el plan de estudios de un simple documento descriptivo a un activo configurable y analizable:

Cuadro 1.6: Analogía de conceptos de SPL en el Dominio Educativo. Fuente: Elaboración propia.

Concepto de SPL	Concepto Análogo en Planificación Curricular	Justificación
Línea de Productos	Carrera o Área de Estudio	El tronco común de conocimientos que define una familia de perfiles profesionales
Producto Específico	Plan de estudio configuracional	La ruta académica única y válida para un estudiante o perfil de egreso definido
Característica (Feature)	Unidad curricular (asignatura, módulo, tema de estudio)	La unidad funcional mínima que puede ser seleccionada u omitida
Característica Raíz	El Currículo Base	La entidad que engloba todos los módulos y asignaturas posibles de la carrera

Continúa en la siguiente página

Cuadro 1.6 – Continuación de la página anterior

Concepto de SPL	Concepto Análogo en Planificación Curricular	Justificación
Variabilidad	Opciones Académicas	El conjunto de decisiones disponibles (optatividad, electividad, elección de menciones) que permiten personalizar la ruta del estudiante

Yendo un paso más adelante para evidenciar la aplicabilidad del Modelo de Características (FM) al dominio curricular, se presenta un ejemplo (figuras 1.2 y 1.3) basado en la asignatura Estructura de Datos I. El modelo computacional interno, que constituye una estructura formal JSON, no se presenta aquí de forma literal por motivos de extensión, pero sí se expone su esquema jerárquico conceptual con el fin ilustrativo y suficiente para comprender las decisiones de variabilidad del FM:

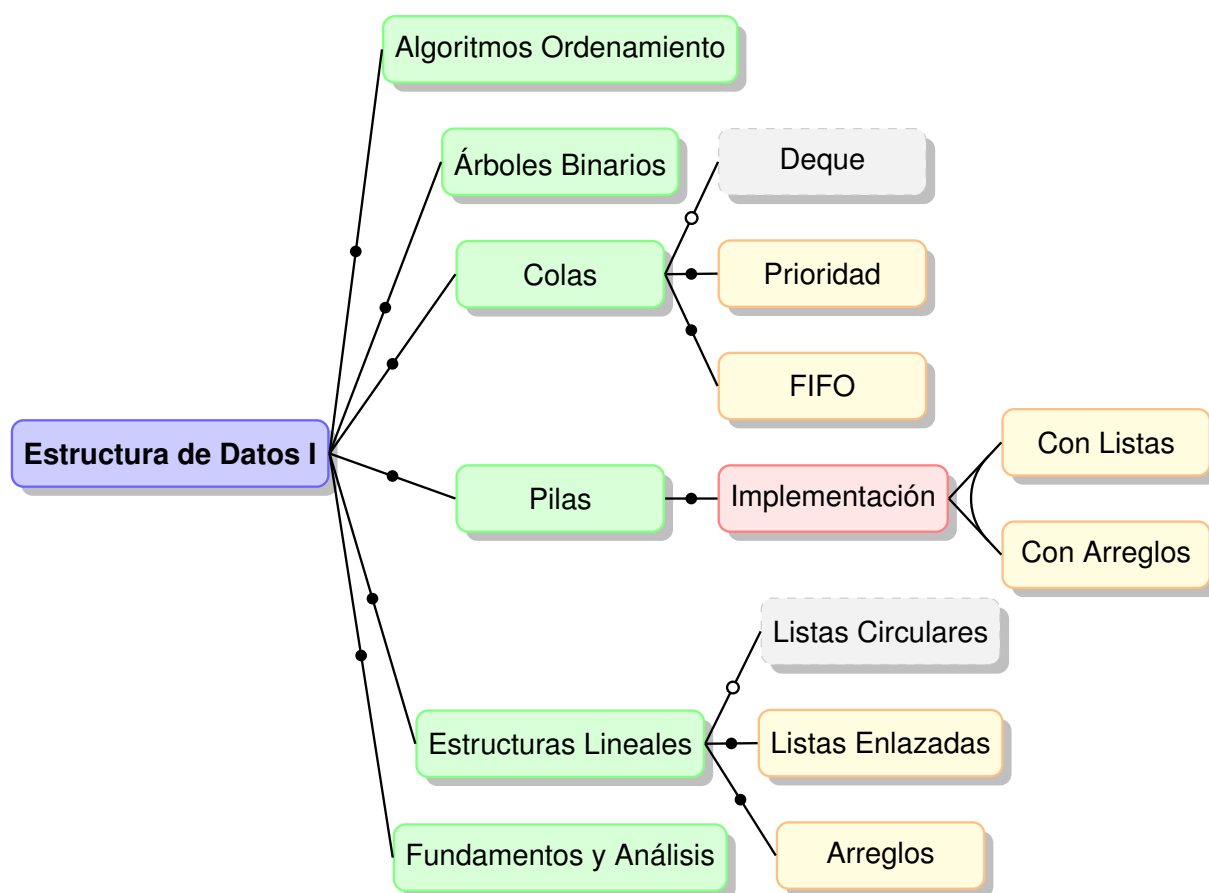


Figura 1.2: Representación simplificada #1 del FM de la asignatura Estructura de Datos I. Fuente: elaboración propia.

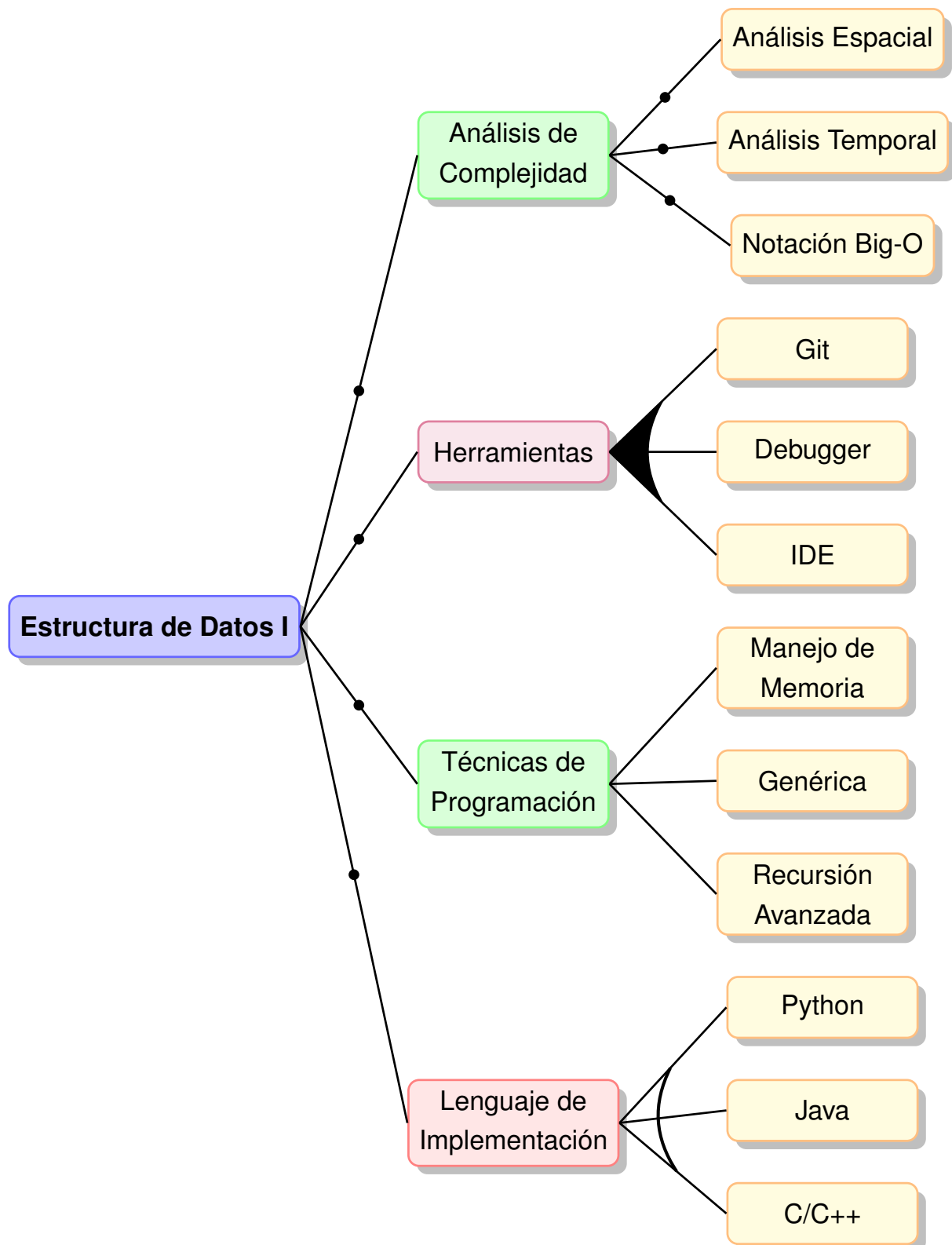


Figura 1.3: Representación simplificada #2 del FM de la asignatura Estructura de Datos I. Fuente: elaboración propia.

Este caso permite demostrar cómo un curso académico, lejos de ser un listado estático de contenidos, posee una estructura jerárquica,

elementos obligatorios y opcionales, variabilidad controlada, dependencias de prerrequisitos y restricciones internas; todas ellas propiedades capturables dentro de un FM formal.

El modelo mostrado constituye una representación declarativa de la asignatura, estructurada como un árbol donde la característica raíz corresponde al curso completo y las características hijas representan módulos, temas o decisiones didácticas. Esta representación no solo captura los contenidos, sino que los organiza con semántica formal, permitiendo análisis automatizable. Los ejemplos ilustra distintos tipos de variabilidad presentes en currículos reales: contenidos obligatorios (módulos troncales), opcionales (tópicos avanzados), alternativas mutuamente excluyentes (implementación de pilas), configuraciones tipo OR (técnicas complementarias) y variantes dependientes (manejo de memoria solo si la implementación se realiza en C/C++).

La estructura del FM muestra claramente cómo una asignatura puede contener subniveles jerárquicos complejos, donde cada nivel constituye un subconjunto configurable. Esto permite modelar asignaturas que se imparten según distintas líneas, enfoques o lenguajes de implementación (C/C++, Java, Python), conservando la validez curricular y evitando inconsistencias.

La variabilidad no solo define qué temas se abordan, sino cómo se implementan, qué carga cognitiva poseen y qué prerrequisitos deben cumplirse. La presencia de relaciones requires, restricciones lógicas e implicaciones capturan dependencias reales, tales como: para comprender árboles es necesario estudiar recursión, o bien optar por C/C++ implica la posibilidad de manejo manual de memoria. Esta lógica curricular, que en los planes tradicionales queda implícita o depende de interpretación docente, aquí se vuelve explícita, formal y verificable.

De este modo, la asignatura deja de ser un documento textual estático para convertirse en un componente configurable. El FM presenta la asignatura como un micro-espacio de variabilidad curricular, análogo

a un “producto configurable” dentro de una línea de productos académicos. Diferentes configuraciones del FM representan distintas adaptaciones válidas de la asignatura según perfiles de estudiante, competencias, niveles de profundidad o enfoques pedagógicos. Este modelo puede procesarse mediante un motor lógico para verificar su consistencia, detectar características inaccesibles (dead features) y generar configuraciones válidas automáticamente.

Este ejemplo demuestra que los Modelos de Características no solo son adecuados para representar planes de estudios completos, sino también para modelar unidades académicas individuales con variabilidad interna. Esto abre la puerta a una flexibilidad curricular controlada, verificable y sustentada en formalismos computacionales, alineándose con los principios de Ingeniería de Líneas de Productos aplicados a la planificación educativa.

1.5.1.3 El Rol Dual de las características (Variabilidad Jerárquica)

Un aspecto clave a formalizar es que la variabilidad no solo ocurre a nivel macro (entre asignaturas), sino también a nivel micro. Un ejemplo de esto, es que una Asignatura actúa como una Característica dentro del árbol curricular global. No obstante, dado que los planes modernos permiten la modularidad y la especialización temática, una Asignatura también puede ser concebida como un Producto Específico que contiene su propia variabilidad interna:

- Nivel 1 (Macro): La Característica "Programación Web" las sub-características "Frontend" (Obligatoria) y
- Nivel 2 (Micro): La Asignatura puede tener "Backend" (Opcional). "Backend" (que es una Característica) puede a su vez ser la Característica Raíz de un sub-FM que modela sus propios temas internos (Ej: OR "Tema: Django" OR "Tema: Spring Boot").

Esta variabilidad jerárquica permite a los gestores modelar con precisión la complejidad de los currículos, desde la estructura de la carrera hasta el desglose temático interno de cada unidad de estudio.

1.5.1.4 Importancia Estratégica y Propósito en la Solución

El uso de los FMs en la planificación curricular no es solo un ejercicio de modelado, sino una ventaja estratégica que permite pasar de la descripción a la comprobación formal del currículo. El propósito del FM en la solución informática propuesta es doble:

- **Estructura de Datos Central:** El FM (codificado en un formato lógico como un árbol JSON) sirve como el modelo de datos maestro del currículo.
- **Base para el Análisis de Consistencia:** La formalidad lógica del FM permite que un motor de análisis (SAT Solver), verifique automáticamente si el plan de estudios modelado contiene inconsistencias (Ej: ciclos de prerrequisitos o restricciones de exclusión que hagan imposible la selección de la Característica Raíz).

1.5.2 Arquitectura conceptual

La arquitectura propuesta se fundamenta en un enfoque modular y basado en componentes, diseñados para soportar la validación, generación y análisis de configuraciones derivadas del Modelo de Características (FM). La estructura conceptual integra elementos propios de arquitecturas estratificadas y de puertos y adaptadores, permitiendo la separación clara entre el núcleo de negocio y los mecanismos de infraestructura. Esta separación es esencial para preservar consistencia, trazabilidad y extensibilidad futura, especialmente en entornos donde los modelos curriculares evolucionan constantemente.

En un nivel superior, el sistema se organiza en tres capas funcionales: presentación, aplicación y dominio, mientras que la capa de infraestructura provee servicios externos que se integran mediante adaptadores. Esta división permite mantener una clara separación de responsabilidades: la API expone servicios REST; la lógica de negocio coordina procesos de validación, generación y análisis; y el dominio encapsula las abstracciones fundamentales del FM (características, restricciones, configuraciones, métricas estructurales, entre otras).

Un elemento central de esta arquitectura lo constituye el subsistema de validación, el cual implementa un enfoque híbrido combinando métodos deterministas (SAT/SMT) con técnicas heurísticas (algoritmos genéticos) y análisis estructural basado en teoría de grafos. Este subsistema se encapsula dentro de la capa de aplicación y se implementa mediante un conjunto de componentes especializados: un validador lógico (PySAT/Z3), un generador de configuraciones y un analizador estructural (NetworkX). Dichos componentes se exponen mediante interfaces formales, facilitando la posibilidad de intercambiar estrategias (SAT, SMT, heurísticas) mediante el patrón Strategy.

Asimismo, la arquitectura conceptual reserva un papel fundamental a los puertos y adaptadores, propios de la Arquitectura Hexagonal (o Clean Architecture). Los puertos definen contratos para interacción hacia adentro (validador, generador, analizador) y hacia afuera (repositorios y persistencia). Los adaptadores implementan las dependencias tecnológicas concretas (PostgreSQL, Redis, MinIO, Z3 Solver, NetworkX, PySAT), permitiendo que el núcleo del sistema permanezca independiente de decisiones de infraestructura.

Desde una perspectiva operativa, la arquitectura soporta escenarios como validación de configuraciones propuestas por usuarios, búsqueda de configuraciones alternativas, análisis automático de errores, diagnóstico de inconsistencias internas del FM, y evaluación de impacto estructural. Adicionalmente, el modelo es extensible a procesos evolutivos

del FM, permitiendo comparar versiones, detectar cambios incompatibles y evaluar decisiones curriculares en términos de validez y redundancia lógica. Esta capacidad de evolución controlada es clave para un soporte curricular robusto y científicamente fundamentado.

1.5.3 Componentes funcionales del sistema

la arquitectura del motor de validación propuesta contempla la separación funcional de tres componentes fundamentales, cada uno respaldado por estrategias tecnológicas que optimizan su ejecución:

1. **Validador Lógico (SAT/SMT Solver):** Responsable de verificar la consistencia global de las decisiones tomadas sobre un FM, incluyendo restricciones booleanas, cardinalidades, relaciones cross-tree y condiciones derivadas. Para esta capa, los enfoques basados en satisfacibilidad (SAT) o satisfacibilidad modulo teorías (SMT) resultan adecuados, dada su capacidad para manejar espacios combinatorios extensos y resolver expresiones complejas con garantías formales. Motores como MiniSAT, Z3 y PySAT han sido ampliamente utilizados en validación de líneas de productos por su eficiencia y soporte para expresiones simbólicas extensas [Benavides et al., 2010a]. Estos mecanismos interactúan con la representación del FM expresada en forma canónica (Forma Normal Conjuntiva (*Conjunctive Normal Form*) (CNF) o Forma Normal Disyuntiva (*Disjunctive Normal Form*) (DNF)), resolviendo satisfacibilidad de manera robusta y escalable.
2. **Generador de Configuraciones (Heurístico / Buscador Guiado):** Este componente se encarga de construir configuraciones válidas a partir del modelo, ya sea para derivar productos académicos completos o proponer alternativas viables ante decisiones parciales proporcionadas por el usuario. Técnicas heurísticas como algoritmos genéticos, búsqueda local o beam search permiten explorar eficientemente espacios grandes sin necesidad de evaluar exhaus-

tivamente todas las combinaciones posibles. Asimismo, el generador puede interactuar con el solver formal: una configuración preliminar producida heurísticamente puede validarse posteriormente mediante SAT/SMT, combinando rapidez exploratoria con corrección lógica.

3. **Analizador Estructural (Estructuras de Grafo y Optimización):**

Encargado de inspeccionar propiedades internas del FM que no dependen únicamente de restricciones lógicas sino de la topología del modelo. Entre estas tareas se incluyen: detección de características inaccesibles (dead features), redundancias, relaciones implícitas, dependencias transitivas, componentes fuertemente conexas y métricas de impacto. Tecnologías como NetworkX, junto con algoritmos clásicos de teoría de grafos (Búsqueda en Profundidad (*Depth-First Search*) (DFS), análisis de caminos, Componentes Fuertemente Conexas (*Strongly Connected Components*) (SCC)), permiten instrumentar estas verificaciones de manera sistemática y automatizada. Este componente resulta indispensable para garantizar la mantenibilidad, comprensibilidad y calidad del FM antes incluso de intentar su configuración.

La combinación de estos tres componentes—validador lógico, generador de configuraciones y analizador estructural—no solo proporciona robustez técnica sino que habilita una plataforma capaz de realizar tareas avanzadas como diagnóstico de inconsistencias, reconfiguración asistida, detección de errores de modelado, proposición de alternativas válidas y verificación formal de rutas académicas. Esta integración híbrida responde adecuadamente al carácter combinatorial del problema, aprovechando solvers exactos cuando la formalidad es crítica y enfoques heurísticos cuando la exploración requiere eficiencia y adaptabilidad.

1.5.4 Patrones de Diseño

La propuesta arquitectónica incorpora un conjunto de patrones de diseño orientados a garantizar extensibilidad, mantenibilidad, independencia tecnológica y coherencia conceptual. La selección de dichos patrones no constituye una decisión meramente ingenieril, sino una fundamentación metodológica que asegura la correcta evolución del sistema, su capacidad para integrar nuevos algoritmos y su adaptabilidad a escenarios curriculares diversos. Desde un punto de vista científico, el uso de patrones sistematiza la estructura de solución, reduce la complejidad accidental e incrementa la robustez verificable del software.

En primer lugar, el patrón **Strategy** resulta esencial para la definición de múltiples mecanismos de validación sobre el Modelo de Características. A través de este patrón, la plataforma puede abstraer e intercambiar, de forma dinámica, estrategias de validación (SAT, SMT, heurística), sin alterar el núcleo de la aplicación ni su modelo de dominio. Esta independencia operativa es particularmente relevante en contextos donde el análisis lógico de configuraciones requiere combinar enfoques deterministas y aproximados, o cuando la evolución de los modelos demanda nuevas técnicas de inferencia formal.

El patrón **Factory Method**, aplicado complementariamente al **Strategy**, permite instanciar validadores, generadores de configuraciones y analizadores estructurales a partir de criterios definidos por la aplicación (por ejemplo, elección entre PySAT y Z3 Solver). De esta manera, la inicialización y configuración de componentes complejos queda encapsulada, asegurando que el núcleo del sistema no dependa de detalles de implementación concreta. Esta abstracción es especialmente útil cuando se integran librerías externas cuyo ciclo de inicialización es no trivial.

Por otra parte, el sistema adopta el patrón **Repository**, encargado de desacoplar el modelo de dominio de los mecanismos de acceso y persistencia en bases de datos. Al emplear interfaces para repositorios y adaptadores para implementaciones (PostgreSQL, Redis, MinIO), la aplicación garantiza independencia tecnológica y capacidad de sustituir

ción futura sin comprometer el modelo conceptual. Este patrón fortalece las propiedades de trazabilidad y versionado del FM, al permitir decidir políticas de almacenamiento, recuperación y auditoría sin afectar la lógica principal.

Adicionalmente, el patrón **Facade** se emplea para unificar operaciones complejas expuestas por el motor de validación. Dado que la validación completa involucra análisis SAT/SMT, heurísticas evolutivas y métricas estructurales, la fachada permite que clientes externos (p. ej., API REST, Interfaz de Línea de Comandos (*Command Line Interface*) (CLI), orquestadores) interactúen con el subsistema de validación a través de una interfaz simplificada, escondiendo la complejidad del proceso interno. Este patrón no solo reduce acoplamiento, sino que también mejora la legibilidad y estabilidad del sistema frente a cambios internos.

El patrón **Observer** se emplea para soportar la evolución del FM. Cuando una característica, restricción o versión del modelo se modifica, componentes dependientes (analizadores estructurales, cachés, herramientas de auditoría) son notificados automáticamente. Este patrón permite evitar inconsistencias temporales, revalidar configuraciones afectadas y disparar procesos de cálculo incremental, preservando coherencia a lo largo del ciclo de vida del modelo.

Complementariamente, la plataforma adopta el patrón **Decorator** para incorporar responsabilidades transversales, tales como caché, autenticación, autorización y trazabilidad, sin modificar el código de los controladores o servicios. Esta técnica habilita una composición funcional altamente flexible, reduciendo duplicación y permitiendo una evolución controlada de dichas funcionalidades auxiliares.

El patrón **Composite** se aplica en la representación jerárquica natural de las características del FM (raíz, subcaracterísticas, alternativas), permitiendo tratar nodos internos y hojas de forma homogénea. Esto simplifica operaciones recursivas de conteo, propagación de restric-

ciones, verificación estructural y evaluación parcial de configuraciones. Su adopción refuerza la estructura intrínseca del problema SPL.

El patrón **Template Method** se utiliza para definir el esqueleto general de algoritmos de persistencia y validación, dejando que pasos específicos se deleguen a las implementaciones concretas (sincrónicas o asíncronas). Este patrón incrementa la reutilización lógica y asegura uniformidad de comportamiento entre componentes funcionalmente equivalentes.

El sistema también emplea el patrón **Chain of Responsibility**, particularmente en la cadena de middlewares para gestión de solicitudes Protocolo de Transferencia de Hipertexto (*Hypertext Transfer Protocol*) (HTTP). Cada componente de la cadena (autorización, verificación, invalidación de caché, auditoría) procesa la solicitud de forma independiente, permitiendo decisiones locales y promoviendo un flujo flexible y extensible. Esta cadena modulariza preocupaciones transversales sin contaminar la lógica del dominio.

En escenarios donde se integran servicios externos (almacenamiento, autenticación, caché distribuida), el patrón **Adapter** garantiza compatibilidad de interfaces y portabilidad tecnológica. La abstracción de clientes externos simplifica testing, reduce acoplamiento y habilita sustitución futura de proveedores sin modificar la lógica del sistema.

Asimismo, la arquitectura incorpora el patrón de **Inversión de Dependencias (Dependency Injection)**, el cual constituye un elemento clave para garantizar reproducibilidad y control del entorno de ejecución. Gracias a este patrón, los componentes que dependen de conexiones a bases de datos, validadores lógicos, servicios de almacenamiento y caché distribuida no instancian sus dependencias directamente, sino que las reciben a través de contenedores o fábricas inyectadas. Esto permite que los algoritmos de validación y generación trabajen sobre interfaces abstractas replicables en distintos contextos (pruebas unitarias, pruebas de carga, depuración, despliegue real), posibilitando reproducir

resultados experimentales y diagnósticos de inconsistencias sin variaciones provocadas por diferencias ambientales. Desde el punto de vista científico, este patrón refuerza la verificabilidad, permitiendo la replicación controlada de experimentos sobre configuraciones FM y asegurando que variaciones en infraestructura no afecten la coherencia de los resultados.

El patrón **Singleton** se aplica para la gestión centralizada de configuraciones globales y recursos compartidos tales como el motor de base de datos, el pool de conexiones y las políticas de validación. Al garantizar la existencia de una única instancia de dichos elementos, el sistema preserva consistencia interna, evita condiciones de carrera y reduce la posibilidad de estados divergentes entre módulos. Este patrón aporta estabilidad operacional, pues asegura que decisiones críticas (como el modo de validación SAT/SMT, parámetros de inicialización simbólica y reglas de auditoría) se mantengan invariantes a lo largo de la ejecución del sistema. Su eficacia es especialmente notable en contextos de ejecución concurrente, donde múltiples agentes interactúan con la plataforma de validación, y constituye una base esencial para la predictibilidad y consistencia del comportamiento computacional.

Finalmente, el patrón **Proxy** se emplea para controlar acceso a recursos sensibles (como la documentación interna y áreas administrativas), aplicando validaciones y restricciones sin duplicar lógica. Esta técnica habilita control fino de acceso y preservación de políticas de seguridad a nivel transversal.

En conjunto, los patrones descritos constituyen la cimentación estructural de la plataforma, contribuyendo a un entorno de validación lógico-analítica de alta confiabilidad, evolutividad y reutilización. Su aplicación metodológica no solo responde a buenas prácticas de ingeniería de software, sino que es también una condición habilitante para cumplir los objetivos científicos de la propuesta, especialmente en lo concerniente a formalización, escalabilidad y reusabilidad del conocimiento curricular

modelado.

1.5.5 Modelo de Datos

El modelo de datos constituye el fundamento estructural a partir del cual se gestionan los elementos curriculares que se modelan mediante Feature Models, así como sus versiones, relaciones, configuraciones válidas y artefactos asociados. Su diseño no responde únicamente a criterios de persistencia, sino a principios formales que garantizan *coherencia semántica, trazabilidad, independencia evolutiva y auditabilidad histórica*.

Desde una perspectiva conceptual, el modelo de datos se orienta hacia la gestión de información versionada, estructuralmente jerárquica y restringida por propiedades lógicas. Esto implica que las entidades no pueden ser tratadas como objetos aislados, sino como componentes interdependientes cuyo comportamiento emergente debe preservarse a lo largo del tiempo. Por ejemplo, una modificación en la estructura de un Feature Model no invalida necesariamente configuraciones existentes, pero debe quedar registrada, versionada y auditada.

En términos metodológicos, el modelo adopta una estructura relacional normalizada, reforzada con mecanismos auxiliares que permiten capturar relaciones n-arias (como constraints), estructuras jerárquicas (feature trees), y vínculos temporales (versioning). Adicionalmente, se incorporan entidades destinadas a soportar procesos analíticos, como el almacenamiento de resultados derivados del análisis SAT/SMT y métricas estructurales.

En la Figura 1.4 se presenta el modelo conceptual relacional adoptado, en el cual se reflejan las principales entidades, sus relaciones cardinales, y las dependencias que permiten soportar tanto la evolución longitudinal del currículo como el proceso computacional de validación.

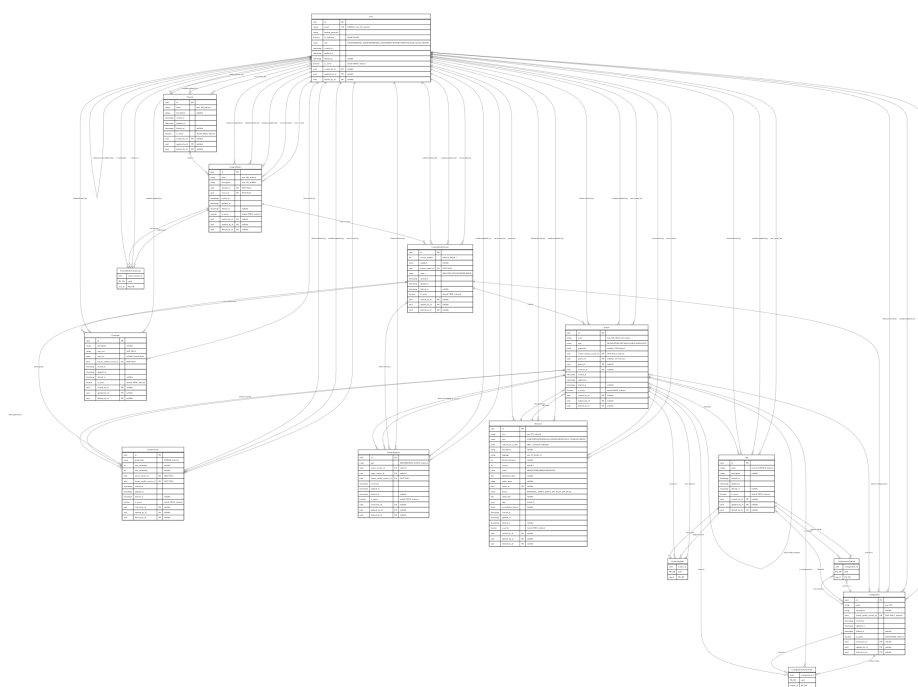


Figura 1.4: Modelo entidad-relación.

Posteriormente se describen las entidades más relevantes, detallando su rol dentro del sistema, los atributos que garantizan trazabilidad y las restricciones que aseguran consistencia operacional y semántica. Se destacan aquellas tablas que poseen un impacto estructural directo sobre la validez de configuraciones y el proceso de inferencia lógica. El modelo comprende las siguientes entidades fundamentales: `feature_model` (Tabla 1.7), que actúa como contenedor maestro; `feature_model_versions` (Tabla 1.8), responsable del versionado inmutable; `features` (Tabla 1.9), que constituye la unidad mínima de aprendizaje; `feature_groups` (Tabla 1.10), para definir puntos de variación; `feature_relations` (Tabla 1.11), que establece dependencias transversales; `constraints` (Tabla 1.12), dedicada a restricciones lógicas formales; y `configurations` (Tabla 1.13), que almacena los itinerarios curriculares derivados.

1.5.5.1 Entidad: Feature Models

Representa las plantillas curriculares maestras (ver Tabla 1.7). Actúa como el contenedor principal de las versiones (Tabla 1.8) y configuraciones (Tabla 1.13).

Cuadro 1.7: Diccionario de datos: Tabla feature_model.

Campo	Tipo	Restricciones	Descripción
id	UUID	PK	Identificador único del modelo.
name	VARCHAR(100)	NOT NULL, INDEXED	Nombre del plan o curso.
description	VARCHAR(255)	NULLABLE	Descripción breve del modelo.
domain_id	UUID	FK (domains), NOT NULL	Dominio de conocimiento asociado.
owner_id	UUID	FK (users), NOT NULL	Propietario/ diseñador principal.
created_at	TIMESTAMP	NOT NULL	Fecha de creación.
updated_at	TIMESTAMP	NULLABLE	Fecha de última actualización.
deleted_at	TIMESTAMP	NULLABLE	Fecha de eliminación lógica.
is_active	BOOLEAN	DEFAULT TRUE	Indica si el registro está activo.
created_by_id	UUID	FK (users), NULLABLE	Usuario que creó el modelo.
updated_by_id	UUID	FK (users), NULLABLE	Usuario que actualizó el modelo.

Continúa en la siguiente página

Cuadro 1.7 – Continuación de la página anterior

Campo	Tipo	Restricciones	Descripción
deleted_by_id	UUID	FK (users), NULLABLE	Usuario que eliminó lógicamente el modelo.

Relaciones:

- Con Domain: domain (N:1) – Asociación al dominio de conocimiento.
- Con User: owner (N:1) – Propietario principal del modelo.
- Con User: collaborators (N:M vía FeatureModelCollaborator) – Colaboradores autorizados.
- Con FeatureModelVersion: versions (1:N) – Historial de versiones del modelo.
- Con User: Relaciones de auditoría (N:1) – Trazabilidad de operaciones CRUD.

1.5.5.2 Entidad: Feature Model Versions

Maneja el versionado de los modelos (ver Tabla 1.8), permitiendo snapshots inmutables y evolución del diseño curricular. Cada versión contiene características (Tabla 1.9), grupos de variabilidad (Tabla 1.10), relaciones cross-tree (Tabla 1.11) y restricciones lógicas (Tabla 1.12).

Cuadro 1.8: Diccionario de datos: Tabla `feature_model_versions`.

Campo	Tipo	Restricciones	Descripción
<code>id</code>	UUID	PK	Identificador único de la versión.
<code>version_number</code>	INTEGER	NOT NULL, INDEXED, DEFAULT 1	Número incremental de versión.
<code>snapshot</code>	JSONB	NULLABLE	Snapshot completo del modelo.
<code>feature_model_id</code>	UUID	FK (feature_model), NOT NULL	Modelo padre.
<code>status</code>	ENUM	DEFAULT 'DRAFT'	Estado: DRAFT, IN_REVIEW, PUBLISHED.
<code>created_at</code>	TIMESTAMP	NOT NULL	Fecha de creación.
<code>updated_at</code>	TIMESTAMP	NULLABLE	Fecha de última actualización.
<code>deleted_at</code>	TIMESTAMP	NULLABLE	Fecha de eliminación lógica.
<code>is_active</code>	BOOLEAN	DEFAULT TRUE	Indica si el registro está activo.
<code>created_by_id</code>	UUID	FK (users), NULLABLE	Usuario que creó esta versión.

Continúa en la siguiente página

Cuadro 1.8 – Continuación de la página anterior

Campo	Tipo	Restricciones	Descripción
updated_by_id	UUID	FK (users), NULLABLE	Usuario que actualizó esta versión.
deleted_by_id	UUID	FK (users), NULLABLE	Usuario que eliminó esta versión.

Relaciones:

- Con FeatureModel: feature_model (N:1) – Versión pertenece a un modelo específico.
- Con Feature: features (1:N) – Características definidas en esta versión.
- Con FeatureGroup: feature_groups (1:N) – Grupos de variabilidad de la versión.
- Con FeatureRelation: feature_relations (1:N) – Relaciones cross-tree.
- Con Constraint: constraints (1:N) – Restricciones lógicas aplicables.
- Con Configuration: configurations (1:N) – Configuraciones derivadas.
- Con User: Relaciones de auditoría (N:1) – Trazabilidad de modificaciones.

1.5.5.3 Entidad: Features

La unidad mínima de aprendizaje (ver Tabla 1.9). Esta tabla es el núcleo del árbol de características, estableciendo relaciones jerárquicas

mediante auto-referencia, pertenencia a grupos (Tabla 1.10) y dependencias transversales (Tabla 1.11).

Cuadro 1.9: Diccionario de datos: Tabla *features*.

Campo	Tipo	Restricciones	Descripción
id	UUID	PK	Identificador único de la feature.
name	VARCHAR(100)	NOT NULL, UK(version_id, name)	Nombre de la característica.
type	ENUM	NOT NULL	MANDATORY, OPTIONAL.
properties	JSONB	NULLABLE, GIN INDEXED	Propiedades extendidas (créditos, horas).
feature_model_version_id	UUID	FK (versions), INDEXED	Versión a la que pertenece.
parent_id	UUID	FK (features), NULLABLE	Jerarquía padre-hijo (auto-referencia).
group_id	UUID	FK (groups), NULLABLE	Grupo XOR-ALTERNATIVE/OR al que pertenece.
resource_id	UUID	FK (resources), NULLABLE	Recurso multimedia asociado.
created_at	TIMESTAMP	NOT NULL	Fecha de creación.

Continúa en la siguiente página

Cuadro 1.9 – Continuación de la página anterior

Campo	Tipo	Restricciones	Descripción
updated_at	TIMESTAMP	NULLABLE	Fecha de última actualización.
deleted_at	TIMESTAMP	NULLABLE	Fecha de eliminación lógica.
is_active	BOOLEAN	DEFAULT TRUE	Indica si el registro está activo.
created_by_id	UUID	FK (users), NULLABLE	Usuario que creó la feature.
updated_by_id	UUID	FK (users), NULLABLE	Usuario que actualizó la feature.
deleted_by_id	UUID	FK (users), NULLABLE	Usuario que eliminó la feature.

Índices:

- **UNIQUE (feature_model_version_id, name)** – Garantiza nombres únicos por versión, evitando duplicación de características dentro del mismo snapshot del modelo.
- **GIN (properties)** – Índice generalizado invertido para búsqueda eficiente en propiedades JSON, permitiendo consultas complejas sobre metadatos curriculares sin deserialización completa.

Relaciones:

- **Jerarquía: parent (N:1), children (1:N)** – Estructura arbórea mediante auto-referencia.
- **Con FeatureGroup: group (N:1), child_groups (1:N)** – Pertenencia a grupos de variabilidad.

- Con Resource: resource (N:1) – Vinculación con recursos educativos multimedia.
- Con Tag: tags (N:M vía FeatureTagLink) – Etiquetado semántico para clasificación.
- Con Configuration: configurations (N:M vía ConfigurationFeatureLink) – Selecciones en itinerarios.
- Con FeatureRelation: source_relations (1:N), target_relations (1:N) – Dependencias transversales.
- Con User: Relaciones de auditoría (N:1) – Trazabilidad de operaciones.

1.5.5.4 Entidad: Feature Groups

Define puntos de variación complejos (ver Tabla 1.10) donde se agrupan varias características hijas (Tabla 1.9) bajo lógica XOR u OR, estableciendo cardinalidades de selección.

Cuadro 1.10: Diccionario de datos: Tabla feature_groups.

Campo	Tipo	Restricciones	Descripción
id	UUID	PK	Identificador único del grupo.
group_type	ENUM	NOT NULL, INDEXED	XOR/ALTERNATIVE (uno y solo uno), OR (al menos uno).
min_cardinality	INTEGER	DEFAULT 1	Mínimo de features a seleccionar.

Continúa en la siguiente página

Cuadro 1.10 – Continuación de la página anterior

Campo	Tipo	Restricciones	Descripción
max_cardinality	INTEGER	NULLABLE	Máximo de features (NULL = sin límite).
parent_feature_id	UUID	FK (features), NOT NULL	Feature que define este grupo.
feature_model_version_id	UUID	FK (versions)	Versión del modelo.
created_at	TIMESTAMP	NOT NULL	Fecha de creación.
updated_at	TIMESTAMP	NULLABLE	Fecha de última actualización.
deleted_at	TIMESTAMP	NULLABLE	Fecha de eliminación lógica.
is_active	BOOLEAN	DEFAULT TRUE	Indica si el registro está activo.
created_by_id	UUID	FK (users), NULLABLE	Usuario que creó el grupo.
updated_by_id	UUID	FK (users), NULLABLE	Usuario que actualizó el grupo.
deleted_by_id	UUID	FK (users), NULLABLE	Usuario que eliminó el grupo.

Relaciones:

- Con Feature: parent_feature (N:1) – Feature que contiene el punto de variación.
- Con Feature: member_features (1:N) – Características hijas que conforman el grupo.
- Con FeatureModelVersion: feature_model_version (N:1) – Versión

del modelo asociada.

- Con User: Relaciones de auditoría (N:1) – Trazabilidad de cambios en grupos.

1.5.5.5 Entidad: Feature Relations

Establece reglas de dependencia transversales (ver Tabla 1.11), conocidas como Cross-Tree Constraints, entre características (Tabla 1.9). Soporta relaciones de tipo "Requiere" (REQUIRES) y "Excluye" (EXCLUDES).

Cuadro 1.11: Diccionario de datos: Tabla `feature_relations`.

Campo	Tipo	Restricciones	Descripción
id	UUID	PK	Identificador único de la relación.
type	ENUM	NOT NULL, INDEXED	REQUIRES (prerequisito), EXCLUDES (incompatible).
source_feature_id	UUID	FK (features), INDEXED	Feature origen (la que depende).
target_feature_id	UUID	FK (features), INDEXED	Feature destino (la requerida/excluida).
feature_model_version_id	UUID	FK (versions)	Versión del modelo.
created_at	TIMESTAMP	NOT NULL	Fecha de creación.

Continúa en la siguiente página

Cuadro 1.11 – Continuación de la página anterior

Campo	Tipo	Restricciones	Descripción
updated_at	TIMESTAMP	NULLABLE	Fecha de última actualización.
deleted_at	TIMESTAMP	NULLABLE	Fecha de eliminación lógica.
is_active	BOOLEAN	DEFAULT TRUE	Indica si el registro está activo.
created_by_id	UUID	FK (users), NULLABLE	Usuario que creó la relación.
updated_by_id	UUID	FK (users), NULLABLE	Usuario que actualizó la relación.
deleted_by_id	UUID	FK (users), NULLABLE	Usuario que eliminó la relación.

Relaciones:

- Con Feature: source_feature (N:1) – Característica que origina la dependencia.
- Con Feature: target_feature (N:1) – Característica destino (requerida o excluida).
- Con FeatureModelVersion: feature_model_version (N:1) – Versión aplicable de la relación.
- Con User: Relaciones de auditoría (N:1) – Registro de modificaciones a restricciones.

1.5.5.6 Entidad: Constraints

Almacena restricciones lógicas complejas o proposicionales (ver Tabla 1.12) que no pueden expresarse como relaciones simples en-

tre características (Tabla 1.11). Soporta expresiones en formato texto y Forma Normal Conjuntiva (CNF) para procesamiento mediante solvers SAT/SMT.

Cuadro 1.12: Diccionario de datos: Tabla `constraints`.

Campo	Tipo	Restricciones	Descripción
id	UUID	PK	Identificador único de la restricción.
description	TEXT	NULLABLE	Descripción en lenguaje natural.
expr_text	TEXT	NOT NULL	Expresión lógica en formato texto.
expr_cnf	JSONB	NULLABLE	Forma Normal Conjuntiva (para SAT solvers).
feature_model_version_id	UUID	FK (versions)	Versión del modelo.
created_at	TIMESTAMP	NOT NULL	Fecha de creación.
updated_at	TIMESTAMP	NULLABLE	Fecha de última actualización.
deleted_at	TIMESTAMP	NULLABLE	Fecha de eliminación lógica.
is_active	BOOLEAN	DEFAULT TRUE	Indica si el registro está activo.
created_by_id	UUID	FK (users), NULLABLE	Usuario que creó la restricción.

Continúa en la siguiente página

Cuadro 1.12 – Continuación de la página anterior

Campo	Tipo	Restricciones	Descripción
updated_by_id	UUID	FK (users), NULLABLE	Usuario que actualizó la restricción.
deleted_by_id	UUID	FK (users), NULLABLE	Usuario que eliminó la restricción.

Relaciones:

- Con `FeatureModelVersion: feature_model_version` (N:1) – Versión del modelo sobre la cual se aplica la restricción lógica.
- Con `User`: Relaciones de auditoría (N:1) – Trazabilidad de definición y modificación de restricciones formales.

1.5.5.7 Entidad: Configurations

Almacena las selecciones específicas realizadas por un usuario (ver Tabla 1.13) sobre una versión de un modelo (Tabla 1.8), representando itinerarios de aprendizaje concretos. Cada configuración vincula un conjunto específico de características seleccionadas (Tabla 1.9) que cumplen las restricciones definidas (Tabla 1.12 y Tabla 1.11).

Cuadro 1.13: Diccionario de datos: Tabla `configurations`.

Campo	Tipo	Restricciones	Descripción
id	UUID	PK	Identificador único de la configuración.
name	VARCHAR(100)	NOT NULL	Nombre del itinerario/plan.

Continúa en la siguiente página

Cuadro 1.13 – Continuación de la página anterior

Campo	Tipo	Restricciones	Descripción
description	TEXT	NULLABLE	Descripción del plan personalizado.
feature_model_version_id	UUID	FK (versions), INDEXED	Versión base del modelo.
created_at	TIMESTAMP	NOT NULL	Fecha de creación.
updated_at	TIMESTAMP	NULLABLE	Fecha de última actualización.
deleted_at	TIMESTAMP	NULLABLE	Fecha de eliminación lógica.
is_active	BOOLEAN	DEFAULT TRUE	Indica si el registro está activo.
created_by_id	UUID	FK (users), NULLABLE	Usuario configurador.
updated_by_id	UUID	FK (users), NULLABLE	Usuario que actualizó la configuración.
deleted_by_id	UUID	FK (users), NULLABLE	Usuario que eliminó la configuración.

Relaciones:

- Con FeatureModelVersion: feature_model_version (N:1) – Versión del modelo a partir de la cual se deriva la configuración.
- Con Feature: features (N:M vía ConfigurationFeatureLink) – Características seleccionadas que componen el itinerario curricular concreto.

- Con `Tag: tags` (N:M vía `ConfigurationTagLink`) – Etiquetas para clasificación y filtrado de configuraciones.
- Con `User`: Relaciones de auditoría (N:1) – Trazabilidad de creación, modificación y eliminación de itinerarios formativos.

Conclusiones

La presente investigación demostró la factibilidad, pertinencia y potencial científico de aplicar la Ingeniería de Líneas de Productos (SPL) a la planificación y gestión curricular en educación superior. El análisis conceptual permitió establecer una analogía formal entre los constructos fundamentales de SPL y las entidades curriculares, revelando que la variabilidad presente en los planes de estudio —en forma de optatividades, prerrequisitos, menciones, orientaciones y rutas formativas— puede ser correctamente modelada como un conjunto estructurado y verificable mediante Modelos de Características (FM). Este hallazgo no solo valida el enfoque metodológico adoptado, sino que además aporta un marco riguroso para la representación y análisis sistemático de la complejidad curricular.

Desde el punto de vista arquitectónico, la propuesta demostró que es viable construir una plataforma computacional que haga uso de FMs versionados, analizados y validados de manera automática, con soporte para configuraciones curriculares personalizadas. Para ello, se integraron técnicas de análisis simbólico, resolución SAT/SMT, heurísticas evolutivas y modelado estructural basado en grafos, garantizando precisión lógica y escalabilidad. La introducción de un modelo híbrido de validación permite abordar escenarios que combinan decisiones obligatorias, alternativas, exclusiones cruzadas y restricciones globales, preservando coherencia y consistencia incluso ante modificaciones evolutivas del currículo.

La arquitectura resultante, fundamentada en patrones de diseño, desacoplamiento conceptual y versionado inmutable, favorece la extensibilidad, la mantenibilidad y la auditabilidad. Asimismo, el almacenamiento estructurado de resultados analíticos, métricas y configura-

ciones abre oportunidades para futuros sistemas inteligentes de recomendación, análisis curricular comparativo e incluso evaluación automatizada de rutas pedagógicas. Los resultados del modelado de datos evidencian que la formalización curricular mediante FMs no constituye un ejercicio teórico aislado, sino una infraestructura aplicable en entornos reales de gestión académica.

Finalmente, la propuesta no pretende sustituir los procesos académicos tradicionales, sino fortalecerlos mediante una abstracción formal que permita control, evidencia verificable y exploración sistemática. Se concluye que la integración de SPL y currículo universitario representa un camino sólido para la modernización de sistemas educativos, habilitando flexibilidad curricular controlada, análisis automatizado, trazabilidad y personalización. Esta contribución abre líneas futuras de investigación orientadas a (i) análisis semántico avanzado de restricciones, (ii) generación automática asistida de itinerarios curriculares óptimos, (iii) interoperabilidad con sistemas institucionales (Sistema de Información Estudiantil (*Student Information System*) (SIS)/Sistema de Gestión del Aprendizaje (*Learning Management System*) (LMS)) y (iv) incorporación de métricas pedagógicas para evaluación dinámica de rutas de aprendizaje.

Lista de Acrónimos

SPL Línea de Productos de Software (*Software Product Line*)

SPLE Ingeniería de Líneas de Productos de Software (*Software Product Line Engineering*)

FM Modelo de Características (*Feature Model*)

FODA Análisis de Dominio Orientado a Características (*Feature-Oriented Domain Analysis*)

SAT Satisfacibilidad Booleana (*Boolean Satisfiability*)

SHARPSAT Conteo de Modelos (*Model Counting*)

META Modelo de Transformación Académica

TPACK Conocimiento Tecnológico Pedagógico del Contenido (*Technological Pedagogical Content Knowledge*)

TIC Tecnologías de la Información y Comunicación

MOOC Curso en Línea Masivo y Abierto (*Massive Open Online Course*)

SIS Sistema de Información Estudiantil (*Student Information System*)

LMS Sistema de Gestión del Aprendizaje (*Learning Management System*)

API Interfaz de Programación de Aplicaciones (*Application Programming Interface*)

XP Extreme Programming

CRC Clase, Responsabilidad y Colaboración

HU Historias de Usuario

CI/CD Integración Continua y Despliegue Continuo

ORM Mapeo Objeto-Relacional (*Object-Relational Mapping*)

SQL Lenguaje de Consulta Estructurado (*Structured Query Language*)

JSON Notación de Objetos JavaScript (*JavaScript Object Notation*)

JSONB JSON Binario (*JSON Binary*)

REST Transferencia de Estado Representacional (*Representational State Transfer*)

ACID Atomicidad, Consistencia, Aislamiento, Durabilidad (*Atomicity, Consistency, Isolation, Durability*)

TTL Tiempo de Vida (*Time To Live*)

RDB Base de Datos Relacional (*Relational Database*)

IDE Entorno de Desarrollo Integrado (*Integrated Development Environment*)

CASE Ingeniería de Software Asistida por Computadora (*Computer-Aided Software Engineering*)

UML Lenguaje de Modelado Unificado (*Unified Modeling Language*)

BPMN Notación de Modelado de Procesos de Negocio (*Business Process Model and Notation*)

HTTP Protocolo de Transferencia de Hipertexto (*Hypertext Transfer Protocol*)

I/O Entrada/Salida (*Input/Output*)

SMT Satisfacibilidad Módulo Teorías (*Satisfiability Modulo Theories*)

CNF Forma Normal Conjuntiva (*Conjunctive Normal Form*)

DNF Forma Normal Disyuntiva (*Disjunctive Normal Form*)

DFS Búsqueda en Profundidad (*Depth-First Search*)

SCC Componentes Fuertemente Conexas (*Strongly Connected Components*)

S3 Simple Storage Service (Amazon S3)

CLI Interfaz de Línea de Comandos (*Command Line Interface*)

Glosario

Variabilidad Curricular

Capacidad de un plan de estudios para ofrecer múltiples rutas de especialización, asignaturas optativas y diferentes prerequisites, permitiendo personalizar la formación académica según las necesidades y objetivos de cada estudiante.

Modelo de Características (*Feature Model*)

Representación jerárquica y estructurada de las características de un sistema o producto, que permite especificar y gestionar la variabilidad mediante relaciones de dependencia, opcionalidad y exclusión entre diferentes componentes o funcionalidades.

Línea de Productos de Software (*Software Product Line*)

Conjunto de sistemas de software que comparten un núcleo común de componentes y se diferencian por características específicas, permitiendo la reutilización sistemática y la gestión eficiente de familias de productos relacionados.

Líneas de Productos Educativos

Aplicación del paradigma de Líneas de Productos de Software al dominio educativo, donde un tronco curricular común puede derivar en múltiples planes de estudio especializados mediante la gestión explícita de su variabilidad.

Sistema Integrado de Gestión Académica (SIGA)

Plataforma institucional diseñada para registrar, administrar y mantener la información académica de estudiantes, programas y procesos administrativos, enfocada en la operación cotidiana más que en el modelado de la variabilidad curricular.

Sistema de Gestión del Aprendizaje (*Learning Management System*)

Plataforma tecnológica que facilita la planificación, entrega y seguimiento de cursos y actividades formativas en línea, permitiendo gestionar contenidos, evaluaciones y la interacción entre docentes y estudiantes.

Servidor de Aplicaciones (*backend*)

Parte del sistema de software que opera en el servidor y se encarga de la lógica de negocio, el procesamiento de datos, la gestión de bases de datos y la comunicación con otros servicios, sin interactuar directamente con el usuario final.

Solución ad hoc (*ad hoc*)

Enfoque diseñado específicamente para un propósito particular, sin seguir un patrón o metodología generalizable, lo que dificulta su reutilización y mantenimiento.

Aula Invertida (*Flipped Classroom*)

Enfoque pedagógico en el que los estudiantes revisan el contenido teórico de forma autónoma (generalmente mediante videos o lecturas) fuera del aula, y utilizan el tiempo en clase para actividades prácticas, discusiones y resolución de problemas con el apoyo del docente.

Microaprendizaje (*Micro-learning*)

Estrategia de aprendizaje que presenta contenidos educativos en pequeñas unidades de información, diseñadas para ser consumidas en períodos cortos de tiempo, facilitando la retención y permitiendo mayor flexibilidad en el proceso de aprendizaje.

Gamificación

Incorporación de elementos y mecánicas propias de los juegos (puntos, niveles, recompensas, desafíos) en contextos educativos para aumentar la motivación, el compromiso (*engagement*) y la participación activa de los estudiantes.

Activos Centrales (*Core Assets*)

Componentes fundamentales (código, arquitectura, modelos de diseño, documentación, planes de prueba) diseñados específicamente para ser compartidos y reutilizados en toda una Línea de Productos de Software. La inversión inicial en estos activos es clave para el éxito de la estrategia de reutilización.

Ingeniería del Dominio (*Domain Engineering*)

Proceso de la Ingeniería de Líneas de Productos de Software enfocado en la creación y el mantenimiento de los Activos Centrales, cuya tarea principal es el modelado de la variabilidad para comprender y documentar todas las posibles configuraciones que la línea puede soportar.

Ingeniería de la Aplicación (*Application Engineering*)

Proceso de la Ingeniería de Líneas de Productos de Software que utiliza los Activos Centrales para configurar y generar productos específicos, mediante la toma de decisiones de variabilidad definidas en el modelo para producir una instancia concreta de la línea de productos.

Puntos de Variabilidad (*Variability Points*)

Ubicaciones explícitas dentro de los Activos Centrales de una Línea de Productos de Software donde se deben tomar decisiones para configurar un producto. Actúan como "interruptores" u "opciones" que guían el proceso de personalización del sistema.

Restricciones Transversales (*cross-tree*)

Reglas lógicas en un Modelo de Características que definen dependencias entre características que no están directamente conectadas en la jerarquía padre-hijo. Capturan relaciones complejas del mundo real como "requiere" (si A entonces B) o "excluye" (A y B no pueden coexistir).

Solucionador SAT (*SAT solver*)

Herramienta algorítmica especializada que determina si una fórmula lógica proposicional puede ser satisfecha (tiene al menos una solución)

válida). En el contexto de Líneas de Productos de Software, se utiliza para verificar si un Modelo de Características tiene configuraciones válidas.

Conteo de Modelos (*Model Counting*)

Proceso de calcular el número total de asignaciones de valores que satisfacen una fórmula lógica. En Modelos de Características, determina cuántas configuraciones de producto válidas existen en una línea de productos, métrica esencial para medir su variabilidad.

Configuración (en SPL)

Proceso sistemático de seleccionar un conjunto específico de características de un Modelo de Características para derivar un producto individual válido, respetando todas las restricciones y dependencias definidas en el modelo.

Motor de Variabilidad Curricular

Componente de software especializado que aplica técnicas de gestión de variabilidad para modelar planes de estudio, validar restricciones pedagógicas y generar configuraciones curriculares personalizadas.

Interfaz de Programación de Aplicaciones (*Application Programming Interface*)

Conjunto documentado de reglas y protocolos que permiten la comunicación entre sistemas de software, facilitando la integración de servicios y la interoperabilidad de aplicaciones.

Configurador Curricular

Herramienta que aplica los principios de las Líneas de Productos de Software al diseño de planes de estudio, guiando la selección de asignaturas y restricciones para generar configuraciones curriculares personalizadas, coherentes y viables.

Diseño Instruccional

Proceso sistemático de planificación, desarrollo e implementación de

experiencias de aprendizaje, basado en teorías pedagógicas y mejores prácticas educativas, con el objetivo de facilitar la adquisición efectiva de conocimientos y habilidades.

Esquema Motor

Estructura cognitiva generalizada que representa un patrón de movimiento aprendido. Según la Teoría del Esquema Motor de Schmidt, la práctica variable enriquece estos esquemas, permitiendo mayor adaptabilidad en diferentes contextos.

Gestión de la Variabilidad

Conjunto de estrategias, procesos y herramientas destinados a identificar, planificar y controlar las diferencias permitidas en productos o planes curriculares para responder a necesidades específicas sin perder coherencia ni calidad.

Análisis de Dominio Orientado a Características (*Feature-Oriented Domain Analysis*)

Metodología pionera introducida por Kang et al. (1990) para el análisis de dominios en ingeniería de software. Formalizó el concepto de Modelos de Características como herramienta principal para capturar la variabilidad funcional en familias de sistemas.

Meta-modelo

Modelo de alto nivel que define los conceptos, relaciones, atributos y restricciones necesarias para especificar formalmente cómo se deben diseñar otros modelos o sistemas. Opera en un nivel de abstracción superior al modelo que describe.

Planificación Curricular

Proceso de diseño y organización sistemática de los elementos que constituyen un plan de estudios: objetivos de aprendizaje, contenidos, metodologías, actividades educativas y criterios de evaluación.

Lógica Proposicional

Sistema formal de lógica que utiliza variables proposicionales (verdadero/-falso) y operadores lógicos (AND, OR, NOT, implica) para representar y analizar relaciones lógicas. Base matemática para el análisis formal de Modelos de Características.

Trayectoria Formativa

Recorrido personalizado de aprendizaje que un estudiante sigue a través de un programa educativo, incluyendo la secuencia de asignaturas, especializaciones y experiencias de aprendizaje seleccionadas según sus intereses y objetivos.

Usabilidad

Grado en que un producto de software puede ser utilizado por usuarios específicos para alcanzar objetivos determinados con efectividad, eficiencia y satisfacción en un contexto de uso específico. Criterio esencial en el diseño de software educativo.

Backend

Capa del sistema que gestiona la lógica de negocio, el procesamiento de datos, la persistencia y las operaciones del servidor. En contraposición al frontend (interfaz visible al usuario), el backend ejecuta procesos no visibles pero esenciales para el funcionamiento del sistema.

Framework

Estructura de software que proporciona un conjunto de herramientas, bibliotecas, patrones y convenciones que facilitan el desarrollo de aplicaciones mediante la reutilización de componentes predefinidos y el establecimiento de estándares arquitectónicos.

Contenedorización

Técnica que permite empaquetar una aplicación junto con todas sus dependencias (bibliotecas, configuraciones, archivos) en unidades aisladas llamadas contenedores, garantizando que se ejecute de forma consistente en cualquier entorno sin problemas de compatibilidad.

Orquestación

Proceso de gestión, coordinación y automatización de múltiples contenedores o servicios distribuidos, asegurando su comunicación, escalabilidad, disponibilidad y resiliencia mediante herramientas especializadas como Docker Compose o Kubernetes.

Workers

Procesos o hilos de ejecución especializados que se encargan de ejecutar tareas específicas de forma paralela o en segundo plano, liberando al proceso principal de operaciones costosas y mejorando la concurrencia y el rendimiento del sistema.

Event Loop

Mecanismo de programación asíncrona que gestiona la ejecución no bloqueante de operaciones I/O, permitiendo que el sistema procese múltiples tareas concurrentemente sin esperar la finalización de operaciones lentas como consultas a base de datos o llamadas de red.

Asincronía

Paradigma de programación donde las operaciones no bloquean la ejecución del programa mientras esperan resultados. Permite que el sistema continúe ejecutando otras tareas mientras espera respuestas de operaciones lentas (I/O, red, disco), mejorando la eficiencia y capacidad de respuesta.

Cache/Caching

Técnica de almacenamiento temporal de datos frecuentemente accedidos en memoria rápida (como RAM), reduciendo la necesidad de consultar fuentes más lentas (disco, base de datos, red) y mejorando significativamente el rendimiento y los tiempos de respuesta del sistema.

Message Broker

Intermediario de software que facilita la comunicación asíncrona entre componentes o servicios mediante el envío, recepción y enrutamiento

de mensajes, permitiendo arquitecturas desacopladas donde los productores y consumidores de información operan de forma independiente.

Snapshots

Instantáneas o capturas del estado completo de un sistema, modelo o conjunto de datos en un momento específico del tiempo. Permiten restaurar versiones anteriores, realizar auditorías históricas o explorar evoluciones temporales sin modificar el estado actual.

Persistencia

Capacidad de un sistema para almacenar datos de forma duradera en medios no volátiles (disco duro, base de datos), garantizando que la información sobreviva a reinicios del sistema, fallos de energía o cierres de aplicación, manteniendo la integridad y disponibilidad de los datos.

PySAT

Biblioteca de Python que proporciona una interfaz unificada para trabajar con diferentes solucionadores SAT (Satisfacibilidad Booleana). Permite codificar problemas lógicos proposicionales y utilizar múltiples motores de satisfacibilidad de manera transparente.

Z3 Solver

Solucionador de teoremas de alto rendimiento desarrollado por Microsoft Research. Capaz de verificar satisfacibilidad de fórmulas lógicas complejas que involucran teorías matemáticas (aritmética, arrays, listas), ampliamente utilizado en verificación formal y análisis de modelos.

MiniSAT

Solucionador SAT minimalista y altamente eficiente que se ha convertido en referencia para el desarrollo de nuevos algoritmos de satisfacibilidad. Destaca por su implementación compacta y rendimiento excepcional en problemas de decisión booleana.

NetworkX

Biblioteca de Python para creación, manipulación y análisis de estructuras de redes complejas y grafos. Proporciona algoritmos para análisis de conectividad, caminos más cortos, componentes, centralidad y otras métricas topológicas fundamentales.

Forma Normal Conjuntiva (CNF)

Representación estándar de fórmulas lógicas proposicionales como conjunción (AND) de disyunciones (OR) de literales. Formato preferido por la mayoría de los solucionadores SAT por su eficiencia computacional y propiedades algorítmicas favorables.

Forma Normal Disyuntiva (DNF)

Representación de fórmulas lógicas proposicionales como disyunción (OR) de conjunciones (AND) de literales. Forma dual a CNF, útil en ciertos contextos de análisis lógico aunque menos común en solucionadores SAT modernos.

Características Muertas (*Dead Features*)

Características en un Modelo de Características que nunca pueden ser seleccionadas en ninguna configuración válida debido a restricciones contradictorias o dependencias imposibles de satisfacer. Su detección es esencial para validar la calidad del modelo.

Algoritmos Genéticos

Técnica de optimización inspirada en la evolución biológica que utiliza operadores de selección, cruce y mutación para explorar espacios de soluciones complejos. Particularmente útil cuando la búsqueda exhaustiva es computacionalmente prohibitiva.

Beam Search

Algoritmo de búsqueda heurística que explora un grafo de estados manteniendo solo un número fijo de mejores candidatos en cada nivel. Balancea exhaustividad y eficiencia, reduciendo el espacio de búsqueda sin sacrificar excesivamente la calidad de soluciones.

PostgreSQL

Sistema de gestión de bases de datos relacional de código abierto altamente extensible, que soporta características avanzadas como tipos de datos personalizados, índices especializados, transacciones ACID completas, replicación y consultas complejas con optimización sofisticada.

Redis

Base de datos en memoria de alto rendimiento que funciona como almacén de estructura de datos (strings, hashes, listas, sets, sorted sets). Ampliamente utilizada para caché, sesiones, colas de mensajes y escenarios que requieren latencia ultra baja.

MinIO

Servidor de almacenamiento de objetos de alto rendimiento compatible con la API de Amazon S3. Diseñado para entornos cloud-native, ofrece almacenamiento distribuido, escalable y resiliente para grandes volúmenes de datos no estructurados.

Búsqueda en Profundidad (*Depth-First Search*)

Algoritmo fundamental de recorrido de grafos que explora tan profundo como sea posible a lo largo de cada rama antes de retroceder. Utilizado para detección de ciclos, análisis de conectividad y ordenamiento topológico.

Componentes Fuertemente Conexas (*Strongly Connected Components*)

Subgrafos maximales de un grafo dirigido donde cada vértice es alcanzable desde cualquier otro vértice dentro del mismo componente. Su identificación es crucial para análisis de dependencias y detección de ciclos en modelos jerárquicos.

Adapter (Adaptador)

Patrón de diseño estructural que permite que interfaces incompatibles trabajen juntas envolviendo un objeto con una interfaz que el cliente

espera. Utilizado para integrar componentes existentes sin modificar su código fuente.

Arquitectura Hexagonal

Patrón arquitectónico que separa la lógica de negocio del núcleo de la aplicación de sus dependencias externas mediante puertos (interfaces) y adaptadores (implementaciones). Promueve independencia tecnológica, testabilidad y evolución controlada del software.

Chain of Responsibility (Cadena de Responsabilidad)

Patrón de diseño conductual que permite pasar solicitudes a lo largo de una cadena de manejadores, donde cada manejador decide si procesa la solicitud o la pasa al siguiente. Utilizado para implementar middlewares y procesamiento secuencial de peticiones.

Clean Architecture

Filosofía de diseño de software que organiza el código en capas concéntricas con dependencias unidireccionales hacia el interior, donde el núcleo de negocio permanece independiente de frameworks, bases de datos e interfaces de usuario, maximizando mantenibilidad y flexibilidad.

Composite (Compuesto)

Patrón de diseño estructural que permite tratar objetos individuales y composiciones de objetos de manera uniforme mediante una estructura de árbol. Ideal para representar jerarquías parte-todo como modelos de características con relaciones padre-hijo.

Decorator (Decorador)

Patrón de diseño estructural que permite agregar responsabilidades adicionales a objetos dinámicamente sin alterar su estructura. Utilizado para implementar funcionalidades transversales como logging, validación y manejo de errores de forma modular.

Pool de Conexiones

Caché de conexiones a bases de datos reutilizables mantenidas en

memoria para reducir el overhead de establecer nuevas conexiones. Mejora significativamente el rendimiento en aplicaciones con alta concurrencia al evitar la creación y destrucción repetida de conexiones costosas.

Dependency Injection (Inversión de Dependencias)

Patrón de diseño que implementa el principio de inversión de dependencias, donde las dependencias de un objeto son proporcionadas externamente en lugar de ser creadas internamente. Facilita el testing, reduce acoplamiento y mejora la modularidad del código.

Factory Method (Método de Fábrica)

Patrón de diseño creacional que define una interfaz para crear objetos pero permite a las subclases decidir qué clase instanciar. Utilizado para delegar la lógica de creación de objetos y facilitar la extensibilidad del sistema.

Façade (Fachada)

Patrón de diseño estructural que proporciona una interfaz simplificada y unificada a un conjunto complejo de subsistemas. Reduce la complejidad percibida y facilita el uso de bibliotecas o frameworks mediante una API más amigable.

Middleware

Componente de software que actúa como intermediario entre diferentes capas o sistemas, procesando solicitudes de forma secuencial en una cadena. En aplicaciones web, se utiliza para implementar autenticación, logging, validación y transformación de datos.

Observer (Observador)

Patrón de diseño conductual que define una dependencia uno-a-muchos entre objetos, de modo que cuando un objeto cambia de estado, todos sus dependientes son notificados automáticamente. Fundamental para implementar sistemas de eventos y arquitecturas reactivas.

Proxy

Patrón de diseño estructural que proporciona un sustituto o representante de otro objeto para controlar el acceso al mismo. Utilizado para implementar carga diferida, control de acceso, logging y caching de forma transparente.

Repository (Repositorio)

Patrón de diseño que encapsula la lógica de acceso a datos proporcionando una interfaz de colección para operaciones CRUD sobre entidades de dominio. Desacopla la lógica de negocio de los detalles de persistencia y facilita el testing con repositorios en memoria.

Singleton

Patrón de diseño creacional que garantiza que una clase tenga una única instancia global y proporciona un punto de acceso global a ella. Utilizado para gestionar recursos compartidos como pools de conexiones, configuraciones y servicios stateful.

Strategy (Estrategia)

Patrón de diseño conductual que define una familia de algoritmos intercambiables encapsulados en clases separadas con una interfaz común. Permite seleccionar el algoritmo en tiempo de ejecución, promoviendo flexibilidad y reutilización.

Template Method (Método Plantilla)

Patrón de diseño conductual que define el esqueleto de un algoritmo en una clase base, permitiendo a las subclasses redefinir ciertos pasos sin cambiar la estructura general. Promueve reutilización de código y establece puntos de extensión controlados.

Versionamiento

Práctica de mantener múltiples versiones de artefactos de software (código, modelos, configuraciones, documentación) con registro histórico de cambios, permitiendo rastrear evolución, revertir modificaciones y gestionar variantes simultáneas del sistema.

Amazon S3 (*Simple Storage Service*)

Servicio de almacenamiento de objetos en la nube de Amazon Web Services que proporciona escalabilidad, disponibilidad de datos, seguridad y rendimiento. Es el estándar de facto para almacenamiento de objetos en la nube, con una API ampliamente adoptada.

Kubernetes

Sistema de orquestación de contenedores de código abierto que automatiza el despliegue, escalado y gestión de aplicaciones en contenedores. Proporciona capacidades avanzadas de auto-recuperación, balanceo de carga, descubrimiento de servicios y gestión de configuración.

Docker Compose

Herramienta para definir y ejecutar aplicaciones Docker multi-contenedor mediante archivos YAML declarativos. Permite especificar servicios, redes, volúmenes y dependencias en un único archivo, simplificando el despliegue y la orquestación de entornos complejos.

On-Premise

Modelo de despliegue de software donde la infraestructura y las aplicaciones se alojan y ejecutan en servidores físicos o virtuales ubicados dentro de las instalaciones de la organización, en contraposición a servicios en la nube. Proporciona mayor control pero requiere inversión en hardware y mantenimiento.

Cloud-Native

Enfoque de diseño y desarrollo de aplicaciones que aprovecha las ventajas de la computación en la nube, como escalabilidad elástica, resiliencia, microservicios y contenedores. Las aplicaciones cloud-native están diseñadas para ejecutarse de forma óptima en entornos de nube distribuidos.

Backtracking

Algoritmo de búsqueda sistemática que explora soluciones potenciales

construyéndolas incrementalmente y abandonando aquellas que no cumplen las restricciones (poda). Ampliamente utilizado en problemas de satisfacibilidad de restricciones y generación de configuraciones válidas.

Réplica

Copia duplicada de datos o servicios mantenida en diferentes ubicaciones o servidores para mejorar disponibilidad, tolerancia a fallos y rendimiento. Las réplicas permiten distribuir la carga de trabajo y garantizar continuidad del servicio ante fallos.

Escalabilidad Horizontal

Capacidad de un sistema para manejar mayor carga agregando más instancias o nodos al sistema, en lugar de aumentar los recursos de una única máquina (escalabilidad vertical). Fundamental para sistemas distribuidos y aplicaciones de alto rendimiento.

Dispersión de Datos (*Data Sharding*)

Técnica de particionamiento horizontal de datos donde un conjunto de datos grande se divide en fragmentos más pequeños distribuidos en múltiples servidores. Mejora el rendimiento, la escalabilidad y permite gestionar volúmenes masivos de información.

Referencias bibliográficas

- AcademiaLab. Línea de productos de software. URL <https://academia-lab.com/enciclopedia/linea-de-productos-de-software/>. Recuperado 25 de octubre de 2025.
- David Adler and Kai Zhang. Evaluating distributed object storage for cloud-native applications: A comparative study of s3-compatible systems. *Journal of Cloud Computing*, 12(4):1–19, 2023.
- Anthology Inc. Blackboard learn: Plataforma para la enseñanza y el aprendizaje, 2025. URL <https://www.anthology.com/blackboard-learn>. Recuperado 25 de octubre de 2025.
- S. Apel, D. Batory, C. Kästner, and G. Saake. *Feature-Oriented Software Product Lines: Concepts and Implementation*. Springer, 2013.
- Poetry authors. Poetry: Python dependency management and packaging. <https://python-poetry.org>, 2024. Accessed: 2025-01-10.
- Mohamed A Awad. A comparison between agile and traditional software development methodologies. *University of Western Australia*, 30:1–69, 2005.
- A. W. Bates. *Teaching in a Digital Age: Guidelines for Designing Teaching and Learning*. Tony Bates Associates Ltd, 2019.
- D. Benavides, S. Segura, and A. Ruiz-Cortés. Automated analysis of feature models 20 years later: A literature review. *Information Systems*, 35(6):615–636, 2010a.
- David Benavides, José-Antonio Trinidad, and Antonio Ruiz-Cortés. A taxonomy of variability constraints for feature models. In *SPLC*, pages 99–108. Springer, 2005.

- David Benavides, Sergio Segura, and Antonio Ruiz-Cortés. Principles of feature modeling. In *Proceedings of the 7th International Conference on Software Product Lines (SPLC)*, pages 13–22. Springer, 2010b.
- David Benavides, Sergio Segura, and Antonio Ruiz-Cortés. Automated analysis of feature models 20 years later: A literature review. In *Information Systems*, volume 35, pages 615–636, 2010c. doi: 10.1016/j.is.2010.01.001.
- Carl Boettiger. An introduction to docker for reproducible research. *ACM SIGOPS Operating Systems Review*, 2015.
- Josiah L Carlson. *Redis in Action*. Manning, 2013.
- CAST. Universal design for learning guidelines version 2.2, 2018. URL <http://udlguidelines.cast.org>.
- L. Chen, M. A. Babar, and N. Ali. Variability management in software product lines: A systematic review. *ACM Computing Surveys*, 53(4): 1–45, 2020.
- Microsoft Corporation. Visual studio code. <https://code.visualstudio.com/>, 2025a. Editor de código fuente multiplataforma, abierto y extensible. Accedido: 2025-12-08.
- Microsoft Corporation. Visual studio code documentation. <https://code.visualstudio.com/Docs>, 2025b. Documentación oficial. Accedido: 2025-12-08.
- Leonardo De Moura and Nikolaj Bjørner. Z3: An efficient SMT solver. In *TACAS 2008 - Tools and Algorithms for the Construction and Analysis of Systems*. Springer, 2008. doi: 10.1007/978-3-540-78800-3_24.
- Ellucian Company. Banner de ellucian. características generales, 2015.
- B. M. Estrada Perea. Definición de un metamodelo basado en usabilidad y conocimiento pedagógico para el diseño de aplicaciones de software educativo. Repositorio Institucional UNAL.

- B. M. Estrada Perea and G. L. Giraldo. Definición de un meta-modelo para el diseño de aplicaciones de software educativo basado en usabilidad y conocimiento pedagógico. *Información Tecnológica*, 33(5), 2022. doi: 10.4067/S0718-07642022000500035.
- Michael Eugene and Robert Anderson. Fastapi framework performance analysis for microservices. *International Journal of Computer Applications*, 2023.
- Erich Gamma, Richard Helm, Ralph Johnson, and John Vlissides. *Design Patterns: Elements of Reusable Object-Oriented Software*. Addison-Wesley, 1995. ISBN 978-0201633610.
- GeeksforGeeks. What is feature engineering? URL <https://www.geeksforgeeks.org/machine-learning/what-is-feature-engineering/>.
- G. M. Ghodke and T. Chavan. An overview of git. *International Journal of Scientific Research in Modern Science and Technology*, 3(6):17–23, 2024.
- YA Gómez. Innovación educativa y gestión curricular. *Anales de la Real Academia de Doctores*. Obtenido de <https://www.rade.es/images-lib/PUBLICACIONES/ARTICULOS/V8N3>, 2, 2023.
- J. F. Guzmán-Luján, J. A. Delgado-Velasco, and R. Amador-Rodezno. Modelado de la variabilidad en foros de discusión para sistemas adaptativos educativos. *Revista Iberoamericana de Tecnologías del Aprendizaje*, 17(1):45–53, 2022.
- Aric A Hagberg, Daniel A Schult, and Pieter J Swart. Exploring network structure, dynamics, and function using networkx. *SC*, 2008.
- J. Hattie. *Visible Learning: A Synthesis of Over 800 Meta-Analyses Relating to Achievement*. Routledge, 2009.

- A. Hernández and E. Rojas. Modelo de transformación académica (meta): una propuesta para la educación superior. *Revista Iberoamericana de Educación*, 80(1):45–62, 2019.
- IBM Corporation. What is an integrated development environment (ide)?, 2023. URL <https://www.ibm.com/think/topics/integrated-development-environment>. Accedido: 10 de diciembre de 2025.
- IEEE. IEEE standard glossary of software engineering terminology, 2014. IEEE Std 610.12-1990.
- Docker Inc. Docker documentation. <https://docs.docker.com/>, 2024a. Accessed: 2025-01-10.
- GitHub Inc. Github documentation. <https://docs.github.com>, 2024b. Accessed: 2025-01-10.
- Postman Inc. Postman api development platform: Documentation. <https://learning.postman.com>, 2024c. Accessed: 2025-01-10.
- Ingeniería de Software. Líneas de producto software para acelerar el desarrollo de aplicaciones relacionadas. URL <https://ingenieriadesoftware.es/lineas-producto-software/>. Recuperado 25 de octubre de 2025.
- Instructure. Canvas lms: Enseñanza y aprendizaje en cualquier lugar, 2025. URL <https://www.instructure.com/canvas>. Recuperado 25 de octubre de 2025.
- Kyo C. Kang, Sholom Cohen, James A. Hess, William E. Novak, and A. Spencer Peterson. Feature-oriented domain analysis (foda) feasibility study. Technical Report CMU/SEI-90-TR-21, Software Engineering Institute, Carnegie Mellon University, 1990.
- Holger Krekel and contributors. pytest documentation. <https://docs.pytest.org>, 2024. Accessed: 2025-01-10.

Jon Loeliger and Matthew McCullough. *Version Control with Git*. O'Reilly, 2012.

Visual Paradigm International Ltd. Visual paradigm documentation. <https://www.visual-paradigm.com/documentation/>, 2024. Accessed: 2025-01-10.

Mark Lutz. Learning python. In *O'Reilly Media*. 2013.

Charlie Marsh. Uv: Extremely fast python package installer. <https://github.com/astral-sh/uv>, 2024. Accessed: 2025-01-10.

M Mendonca and A Wasowski. Sat-based analysis of feature models is easy. In *SPLC*, pages 231–240. ACM, 2009.

Aaron et al. Meurer. Sympy: symbolic computing in python. *PeerJ Computer Science*, 2017.

MinIO Documentation. MinIO Inc., 2024. URL <https://min.io/docs>. Accessed: 2025-01-15.

P. Mishra and M. J. Koehler. Technological pedagogical content knowledge: A framework for teacher knowledge. *Teachers College Record*, 108(6):1017–1054, 2006.

Bruce Momjian. *PostgreSQL: Introduction and Concepts*. Addison-Wesley, 2001.

Moodle Project. ¿qué es moodle? visión general de la plataforma, 2025. URL <https://moodle.com/es/what-is-moodle/>. Recuperado 25 de octubre de 2025.

Unified Modeling Language (UML) Specification. Object Management Group, 2017. URL <https://www.omg.org/spec/UML>. Accedido: 10 de diciembre de 2025.

J. Pacienza. *Metodologías de desarrollo de software*. UCA, 2015.

- Luigi Pesce. Performance benchmarking of python web frameworks. *Journal of Systems and Software*, 2021.
- K. Pohl, G. Böckle, and F. J. van der Linden. *Software Product Line Engineering: Foundations, Principles and Techniques*. Springer, 2005.
- P. K. Ponuthurai and J. Loeliger. *Version Control with Git*. O'Reilly Media, 2022.
- PostgreSQL Global Development Group. Postgresql official documentation. <https://www.postgresql.org/docs/>, 2024. Accessed: 2025-01-10.
- R. S. Pressman. *Software Engineering: A Practitioner's Approach*. McGraw-Hill Education, 8 edition, 2014.
- Celery Project. Celery distributed task queue documentation. <https://docs.celeryq.dev>, 2024. Accessed: 2025-01-10.
- PTC. Feature models and features: What are they, 2025. URL <https://www.ptc.com/en/blogs/alm/modelling-features>. Recuperado 25 de octubre de 2025.
- pure-systems GmbH. pure::variants product overview, 2024. URL <https://www.pure-systems.com/en/products/purevariants>. Recuperado 25 de octubre de 2025.
- David Ramel. New open source index: Vs code is no. 1 code editor. *Visual Studio Magazine*, 2021.
- Sebastián Ramírez. Fastapi: Modern, fast (high-performance) web framework for building apis with python 3.6+. <https://fastapi.tiangolo.com>, 2019. Accessed: 2025-01-10.
- L. Raviv, G. Lupyan, and S. C. Green. How variability shapes learning and generalization. *Trends in Cognitive Sciences*, 26(6):462–483, 2022.

- Baishakhi Ray, Daryl Posnett, Vladimir Filkov, and Premkumar Devanbu. A large-scale study of programming languages and code quality in github. *Communications of the ACM*, 2014.
- M. Richards. *Software Architecture Patterns*. O'Reilly Media, 2015.
- P. Rodríguez, A. Vizcaíno, and M. Piattini. Herramientas de gestión curricular: Una evaluación de la situación actual. *IEEE Revista Iberoamericana de Tecnologías del Aprendizaje*, 14(3):112–120, 2019.
- D. H. Rose and N. Strangman. Universal design for learning: Meeting the challenge of individual learning differences through a neurocognitive perspective. *Universal Access in the Information Society*, 5(4):381–391, 2007.
- James Rumbaugh, Ivar Jacobson, and Grady Booch. *Unified Modeling Language Reference Manual*. Pearson, 2004.
- R. A. Schmidt. A schema theory of discrete motor skill learning. *Psychological Review*, 82(4):225–260, 1975.
- Robert W. Sebesta. *Concepts of Programming Languages*. Pearson, 11 edition, 2016. ISBN 978-0133943023.
- SG Buzz. Líneas de productos de software. URL <https://sg.com.mx/revista/34/lineas-productos-software>. Recuperado 25 de octubre de 2025.
- I. Sommerville. *Software Engineering*. Pearson Education, 10 edition, 2016.
- Mate Soos, Stephan Gocht, and et al. Ganak: A scalable probabilistic model counter. In *SAT Conference*. Springer, 2020.
- C. Sundermann, L. Dörr, M. Junker, and T. Kehler. Evaluating state-of-the-art #sat solvers on industrial configuration spaces. *Empirical Software Engineering*, 28(29), 2023.

- J. Sánchez, F. L. Gutiérrez, and M. Cabrera. Hacia una metodología para la gestión de la variabilidad en planes de estudio de ingeniería de software. *Journal of Educational Computing Research*, 58(5):1020–1045, 2020.
- T. Thüm, C. Kästner, F. Benduhn, J. Meinicke, G. Saake, and T. Leich. Featureide: An extensible framework for feature-oriented software development. *Science of Computer Programming*, 79:70–85, 2014.
- UNESCO. *Reimagining our futures together: A new social contract for education*. UNESCO, 2022.
- Guido Van Rossum and Fred L Drake. *Python Tutorial*. Centrum voor Wiskunde en Informatica (CWI), 1995.
- Hongyu Wang, Lu Zhao, and Jun Hu. Formal semantics and verification for feature modeling. *Computer Software and Applications Conference (COMPSAC)*, pages 148–155, 2007.
- J. Weinstein. Liderazgo directivo y gestión de la diversidad. In *Calidad en la educación: Una mirada desde la gestión de la diversidad*, pages 15–42. Ministerio de Educación, 2002.
- Han Zhao, Michael Liffiton, Patrick Eijk, Alexey Malikov, and Alessandro Previti. PySAT: A Python toolkit for SAT-based prototyping. In *SAT 2018 - 21st International Conference on Theory and Applications of Satisfiability Testing*. Springer, 2018. doi: 10.1007/978-3-319-94144-8_38.